TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN – ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN 1

BLOXORZ SOLVER

Môn học: Cơ sở Trí tuệ nhân tạo
Mã môn học: CSC14003

Giảng viên phụ trách	GS.TS Lê Hoài Bắc ThS Nguyễn Ngọc Đức
Nhóm – Lớp	24TNT1
Ngày nộp	Ngày 1 tháng 11 năm 2026
Source Code	hoanghiep06/Bloxorz_Solver
Video Demo	Xem Video Demo trên YouTube

TP. Hồ Chí Minh – 2026

Tóm tắt

Báo cáo trình bày quá trình mô hình hóa và giải bài toán Bloxorz dưới dạng bài toán tìm kiếm trạng thái trong trí tuệ nhân tạo. Mô hình trạng thái biểu diễn vị trí và tư thế của khối hộp, trạng thái các cầu, cũng như cơ chế tách và hợp nhất hai khối đơn vị. Mô hình chuyển trạng thái xử lý các luật di chuyển, điều kiện hợp lệ và những cơ chế đặc biệt như ô dễ vỡ, công tắc và cầu đóng/mở. Trên cơ sở đó, nhóm cài đặt và khảo sát các thuật toán Breadth-First Search (BFS), Depth-First Search (DFS), Iterative Deepening Search (IDS), Uniform-Cost Search (UCS), A* và Greedy Best-First Search (GBFS). Trong đó, GBFS được bổ sung nhằm mở rộng phạm vi so sánh giữa các chiến lược tìm kiếm không thông tin, tìm kiếm theo chi phí và tìm kiếm có định hướng bằng heuristic.

Phần thực nghiệm đánh giá các thuật toán thông qua thời gian tìm kiếm, bộ nhớ sử dụng, số trạng thái mở rộng và độ dài lời giải. Bên cạnh việc so sánh hiệu năng, nhóm khảo sát ảnh hưởng của các trọng số trong hàm chi phí UCS và hàm heuristic của A*, GBFS thông qua tìm kiếm tham số (Grid Search) kết hợp đánh giá chéo theo từng màn chơi. Kết quả cho thấy việc tinh chỉnh tham số không nhất thiết cải thiện hiệu quả tổng quát so với cấu hình ban đầu: UCS và GBFS giữ cấu hình baseline khi các cấu hình thay thế chưa cho thấy sự cải thiện đủ thuyết phục trên các tập kiểm định. Đối với A*, hệ số heuristic được lựa chọn giúp giảm số trạng thái mở rộng trung bình khoảng 23% so với trường hợp không sử dụng heuristic trên bộ 18 màn chơi khảo sát. Tuy nhiên, kết quả thực nghiệm này không thay thế việc kiểm chứng các tính chất lý thuyết của heuristic, đặc biệt đối với những trạng thái có cơ chế tách khối. Báo cáo phân tích ưu điểm, hạn chế của từng thuật toán và đề xuất việc xây dựng heuristic phù hợp hơn cho trạng thái tách khối như một hướng cải tiến tiếp theo.

Mục lục

Tóm tắt **i**

Danh sách bảng **v**

Danh sách hình **vi**

1 Tổng quan dự án và tái hiện trò chơi **1**

1.1 Bối cảnh và mục tiêu . . . 1
1.2 Đặc tính bài toán . . . 1
1.3 Bàn chơi và quy ước mã ô . . . 1
1.4 Khối và các tư thế cơ bản . . . 1
1.5 Các cơ chế nâng cao . . . 2
1.5.1 Ô dễ vỡ . . . 2
1.5.2 Cầu và công tắc . . . 2
1.5.3 Tách và nhập lại khối . . . 3
1.6 Kiến trúc phần mềm . . . 3

2 Mô hình hóa bài toán và thuật toán tìm kiếm **5**

2.1 Phát biểu bài toán tìm kiếm . . . 5
2.2 Biểu diễn trạng thái . . . 5
2.2.1 Khối nguyên vẹn . . . 5
2.2.2 Chế độ tách khối . . . 5
2.3 Tập hành động, chuyển trạng thái và điều kiện đích . . . 6
2.3.1 Tập hành động . . . 6
2.3.2 Bảng chuyển động hình học . . . 6
2.3.3 Điều kiện hợp lệ và tác dụng của công tắc . . . 7
2.3.4 Trạng thái ban đầu và kiểm tra đích . . . 7
2.4 Breadth-First Search (BFS) . . . 8
2.5 Depth-First Search (DFS) . . . 8
2.6 Iterative Deepening Search (IDS) . . . 9
2.7 Uniform-Cost Search (UCS) . . . 9
2.7.1 Các mô hình chi phí có trong mã nguồn . . . 11
2.7.2 Hàm chi phí tổng hợp được dùng trong thực nghiệm . . . 11
2.8 A* Search . . . 12
2.8.1 Heuristic nền được triển khai . . . 12

2.8.2 Tinh chỉnh hệ số heuristic . . . 13
2.8.3 Tính chất và giới hạn của heuristic . . . 13
2.8.4 Pseudocode của A* . . . 14
2.9 Greedy Best-First Search (GBFS) – phương pháp bổ sung . . . 15
2.9.1 Heuristic và các tham số . . . 15
2.9.2 Pseudocode . . . 16
2.9.3 Giới hạn và vai trò trong thực nghiệm . . . 16
2.10 Tính chất và độ phức tạp . . . 17
2.11 Quy ước đếm node và đo đạc trong chương trình . . . 17

3 Thiết lập thực nghiệm và đánh giá 19
3.1 Mục tiêu và nguyên tắc thực nghiệm . . . 19
3.2 Môi trường và quy trình chạy . . . 19
3.3 Mô tả Chi tiết Thiết kế từng Màn chơi . . . 21
3.3.1 Level 01: Nhập môn Hình học Không gian (Tutorial Basics) . . . 21
3.3.2 Level 02: Khúc eo Gấp khúc & Ngõ cụt (Narrow Passages) . . . 21
3.3.3 Level 03: Vòng xoay Thay đổi Tư thế (Parity Inversion Island) . . . 21
3.3.4 Level 04: Mê cung Diện tích Lớn (Open Field Labyrinth) . . . 21
3.3.5 Level 05: Vùng cấm Ô cam (Fragile Introduction) . . . 21
3.3.6 Level 06: Bẫy Sập Sàn Đôi (Dual Fragile Trap) . . . 22
3.3.7 Level 07: Thao trường Ô cam Dày đặc (Fragile Runway) . . . 22
3.3.8 Level 08: Vực thẳm Đa chướng ngại (Complex Fragile Obstacles) . . . 22
3.3.9 Level 09: Cầu nối Nhị phân và Công tắc Tròn (Soft Switch Mechanics) . . . 22
3.3.10 Level 10: Thử thách Áp lực Công tắc Chữ X (Heavy Switch Prerequisite) 22
3.3.11 Level 11: Kết hợp Cầu động và Ô cam Sập sàn (Bridges with Fragile Hazard) 23
3.3.12 Level 12: Bãi mìn Ô cam và Nút chữ X (Heavy Switch over Fragile Field) 23
3.3.13 Level 13: Hệ thống Cầu kép Đối xứng (Dual Bridge Navigation) . . . 23
3.3.14 Level 14: Công tắc Phức hợp Đa chức năng (Multi-Switch Logic Puzzle) . 23
3.3.15 Level 15: Phân thân Khối gỗ (Split Block Introduction) . . . 24
3.3.16 Level 16: Mê cung Vực thẳm Cực hạn (The Marathon Chokepoint) . . . 24
3.3.17 Level 17: Phân đoạn Đảo Cô lập và Bẫy Ô cam (Island Hopping) . . . 24
3.3.18 Level 18: Chung kết Đỉnh cao – Tổng lực Cơ chế (Grand Finale Masterpiece) 24
3.4 Chỉ số đánh giá và cách đo . . . 24
3.5 Kết quả benchmark . . . 26
3.5.1 Độ dài lời giải theo từng level . . . 26
3.5.2 So sánh tài nguyên tính toán . . . 28
3.6 Giới hạn thực nghiệm và tính tái lập . . . 30
3.7 Quy trình lựa chọn tham số cho UCS, A* và GBFS . . . 30

4 Kế hoạch và phân công công việc **33**

4.1 Phân công nhiệm vụ 33

5 Kết luận **34**

5.1 Kết quả và đóng góp trong phạm vi đồ án 34

Tài liệu tham khảo **35**

Danh sách bảng

1.1 Quy ước mã ô trong bộ level 2
1.2 Các lớp chức năng chính trong source code 3

2.1 Bảng chuyển động của khối nguyên vẹn 6
2.2 Bảng chuyển động trong chế độ tách khối (Split mode) 7
2.3 Tính chất lý thuyết và lưu ý triển khai 18

3.1 Môi trường thực thi và thông số máy kiểm nghiệm benchmark 20
3.2 Định nghĩa chỉ số benchmark và đánh giá chất lượng 25
3.3 Tổng hợp kết quả benchmark trung bình trên 18 màn chơi 26
3.4 Số action giải được theo từng level và solver (18 màn chơi) 27
3.5 Kết quả khảo sát hệ số heuristic α của A* trên 18 level 31
3.6 Tổng hợp kết quả lựa chọn tham số 32

4.1 Phân công nhiệm vụ chính của các thành viên 33

Danh sách hình vẽ

1.1 Giao diện đồ họa của Bloxorz Solver 4

3.1 So sánh tài nguyên tìm kiếm. Mỗi cột là giá trị trung bình trên 18 màn chơi sau khi lấy trung bình của 5 lượt lặp độc lập cho mỗi màn. 28

3.2 Bộ nhớ RAM đỉnh trung bình (KiB) theo thuật toán. Chỉ số là đỉnh cấp phát Python được truy vết thông qua thư viện `tracemalloc`, không phải RSS hệ điều hành. 29

CHƯƠNG 1

Tổng quan dự án và tái hiện trò chơi

1.1 Bối cảnh và mục tiêu

Bloxorz là trò chơi giải đố trong đó người chơi điều khiển một khối hộp chữ nhật kích thước $1 \times 1 \times 2$ lăn trên một bàn chơi dạng lưới. Mục tiêu là đưa khối hộp nguyên vẹn đứng thẳng vào ô đích. Bài toán phù hợp để nghiên cứu tìm kiếm vì trạng thái phụ thuộc không chỉ vào vị trí mà còn vào tư thế khối, và các cơ chế cầu/công tắc có thể làm thay đổi tập hành động hợp lệ theo từng thời điểm [3, 1].

Mục tiêu của dự án là: (i) xây dựng phiên bản có giao diện đồ họa để chơi Bloxorz; (ii) mô hình hóa trò chơi thành bài toán tìm kiếm trạng thái; (iii) cài đặt các thuật toán tìm kiếm theo yêu cầu môn học; và (iv) thiết kế thực nghiệm trên cùng một tập level để so sánh lời giải và tài nguyên tính toán.

1.2 Đặc tính bài toán

Bài toán có các đặc tính sau:

- **Quan sát đầy đủ:** cấu trúc level và trạng thái hiện tại được mô tả tường minh trong dữ liệu trò chơi.
- **Xác định:** với một trạng thái và một hành động hợp lệ, hàm chuyển trạng thái trả về nhiều nhất một trạng thái kế tiếp.
- **Rời rạc:** vị trí, tư thế, trạng thái cầu và hành động được biểu diễn bằng các giá trị rời rạc.
- **Không gian trạng thái hữu hạn với một level hữu hạn:** số tọa độ, tư thế, cấu hình cầu và tổ hợp vị trí hai khối đơn vị đều hữu hạn; do đó có thể sử dụng cơ chế ghi nhận trạng thái để tránh chu trình.

1.3 Bàn chơi và quy ước mã ô

Level được đọc từ tệp JSON. Bàn chơi có R hàng và C cột; mỗi ô chứa mã tile. Bảng 1.1 tóm tắt quy ước mã hóa được sử dụng trong các level đi kèm source code.

1.4 Khối và các tư thế cơ bản

Khi khối chưa bị tách, trạng thái hình học thuộc một trong ba kiểu:

- **Đứng thẳng (STANDING):** khối chiếm một ô (r, c).
- **Nằm ngang (HORIZONTAL):** khối chiếm (r, c) và $(r, c + 1)$.

Bảng 1.1: Quy ước mã ô trong bộ level

Mã	Loại ô	Ý nghĩa trong mô hình
`0`	Vực / không có sàn	Ô không hợp lệ; khối không thể chiếm giữ ô này.
`1`	Sàn thường	Có thể hỗ trợ khối nếu nằm trong biên bàn chơi.
`2`	Ô đích	Là tọa độ mục tiêu; chỉ được công nhận khi khối nguyên vẹn đứng thẳng tại ô này.
`3`	Ô dễ vỡ (fragile)	Khối nguyên vẹn không được đứng thẳng trên ô này; tư thế nằm được phép. Khối đơn vị trong chế độ split được xử lý riêng.
`4`	Soft switch	Có thể được kích hoạt khi ô nằm dưới bất kỳ phần nào của khối nguyên vẹn, hoặc khi khối đơn vị đi vào ô đó.
`5`	Heavy switch	Chỉ được kích hoạt khi khối nguyên vẹn đứng thẳng tại ô công tắc.
`6`	Split switch	Khi được tiếp cận ở tư thế đứng thẳng, chuyển khối thành hai khối đơn vị và đặt chúng tại các tọa độ được cấu hình trong level.
`7`	Ô cầu (bridge)	Tính hợp lệ phụ thuộc cờ trạng thái cầu tương ứng; cầu đóng được xử lý như vực, cầu mở có thể đi qua.

- **Nằm dọc (VERTICAL):** khối chiếm (r, c) và $(r + 1, c)$.

Tọa độ (r, c) là ô neo: với tư thế nằm, đó là ô phía trên hoặc ô bên trái trong hai ô đang chiếm giữ. Quy ước này được dùng nhất quán trong bảng chuyển trạng thái ở Chương 2.

Đối với khối nguyên vẹn, trạng thái tìm kiếm được xác định bởi vị trí neo, tư thế khối và cấu hình cầu hiện tại. Khi khối bị tách, trạng thái cần lưu vị trí của cả hai khối đơn vị và chỉ số khối đang được điều khiển, bên cạnh cấu hình cầu. Có thể biểu diễn khái quát:

$$S_{\text{whole}} = (r, c, o, \mathbf{b}), \qquad S_{\text{split}} = (p_0, p_1, k, \mathbf{b}),$$

trong đó (r, c) là tọa độ neo, o là tư thế, p_0, p_1 là vị trí hai khối đơn vị, k là khối đang được điều khiển và $\mathbf{b}$ là vector trạng thái các cầu. Biểu diễn này giúp phân biệt những trạng thái có cùng vị trí khối nhưng khác tư thế, quyền điều khiển hoặc cấu hình cầu.

1.5 Các cơ chế nâng cao

1.5.1 Ô dễ vỡ

Trong source, một trạng thái của khối nguyên vẹn không hợp lệ nếu khối đứng thẳng trên ô fragile. Khi khối nằm ngang hoặc nằm dọc, cả hai ô chiếm giữ được kiểm tra tính hợp lệ và cơ chế fragile không loại trạng thái chỉ vì có ô dễ vỡ. Trong chế độ split, hai khối đơn vị được kiểm tra theo luật riêng và có thể đi trên ô fragile.

1.5.2 Cầu và công tắc

Mỗi cầu có một cờ Boolean trong tuple `bridges_state`: giá trị `True` biểu thị cầu mở và đi được; giá trị `False` biểu thị cầu đóng và không đi được. Mỗi công tắc có thể tác động lên một hoặc

nhiều cầu theo cấu hình trong JSON. Source hỗ trợ các kiểu tác động `toggle`, `open/permanent` và `close`; với dữ liệu hiện tại, `open` hoặc `permanent` đặt trạng thái cầu thành mở, còn `toggle` đảo trạng thái cờ.

Soft switch được xét khi khối nguyên vẹn đứng hoặc nằm, cũng như khi một khối đơn vị đi vào ô công tắc. Heavy switch chỉ được xét khi khối nguyên vẹn đứng thẳng; hai khối đơn vị trong chế độ split không kích hoạt heavy switch. Trạng thái cầu sau khi kích hoạt công tắc được lưu trong trạng thái tìm kiếm, vì vậy hai trạng thái có cùng vị trí khối nhưng cấu hình cầu khác nhau không được xem là một trạng thái.

1.5.3 Tách và nhập lại khối

Khi khối nguyên vẹn đứng thẳng tại split switch có cấu hình hợp lệ, trò chơi dịch chuyển hai khối đơn vị đến hai tọa độ được khai báo trong trường `split_positions`. Mỗi khối đơn vị có thể di chuyển bằng các hướng `U`, `D`, `L`, `R`; hành động `SPACE` đổi khối đang được điều khiển. Hai khối không được chiếm cùng một ô. Khi khối đang điều khiển di chuyển tới vị trí kề trực giao với khối còn lại, bộ máy trò chơi nhập chúng trở lại thành khối $1 \times 1 \times 2$. Trạng thái split không bao giờ thỏa điều kiện chiến thắng.

1.6 Kiến trúc phần mềm

Source code được tổ chức theo các lớp chức năng chính như Bảng 1.2. Giao diện được xây dựng bằng Python và Panda3D; mô hình trò chơi và thuật toán tìm kiếm được tách khỏi mã hiển thị để có thể kiểm thử bằng terminal mà không cần chạy renderer.

Bảng 1.2: Các lớp chức năng chính trong source code

Thành phần	Tệp/thư mục	Vai trò
Core state	`src/core/state.py`	Kiểu trạng thái bất biến, tư thế khối, hai tọa độ khối con, khối đang điều khiển và quy tắc so sánh/hash trạng thái.
Core board	`src/core/board.py`	Nạp JSON, xác thực ô/trạng thái, sinh hành động, xử lý chuyển trạng thái, công tắc, cầu và split/rejoin.
Search solvers	`src/solvers/`	BFS, DFS, IDS, A*, GBFS và lớp UCS cùng các mô hình chi phí.
GUI	`src/gui/`	Renderer 3D, camera, HUD, điều khiển, animation và kết nối solver với giao diện.
Levels	`levels/*.json`	Dữ liệu bàn chơi; cho phép tái sử dụng cùng level giữa các solver.
Tests	`tests/`, `tools/`	Kiểm thử logic trạng thái, bàn chơi, solver, hợp đồng GUI và các kịch bản tích hợp.



Hình 1.1: Giao diện đồ họa của Bloxorz Solver

CHƯƠNG 2

Mô hình hóa bài toán và thuật toán tìm kiếm

2.1 Phát biểu bài toán tìm kiếm

Có thể mô hình hóa một level Bloxorz dưới dạng bộ

$$\mathcal{P} = \langle \mathcal{S}, \mathcal{A}, T, S_0, \mathcal{G}, c \rangle, \tag{2.1}$$

trong đó $\mathcal{S}$ là tập trạng thái hợp lệ; $\mathcal{A}$ là tập hành động có thể áp dụng; T là mô hình chuyển trạng thái; S_0 là trạng thái đầu; $\mathcal{G}$ là tập trạng thái đích; và $c(S, a, S')$ là chi phí khi thực hiện hành động a từ S sang S'. Độ dài danh sách hành động mà solver trả về tính cả `SPACE` nếu hành động này xuất hiện trong chế độ Split. Khi báo cáo số lần lăn khối, chỉ đếm U, D, L, R và tách riêng thao tác đổi khối điều khiển. UCS và A* tối ưu tổng chi phí theo cost model đang sử dụng, không nhất thiết tối ưu số hành động hoặc số lần lăn; GBFS chỉ dùng heuristic để sắp xếp frontier.

2.2 Biểu diễn trạng thái

2.2.1 Khối nguyên vẹn

Khi khối chưa bị tách, trạng thái có dạng

$$S_{\text{whole}} = (r, c, o, \mathbf{b}), \tag{2.2}$$

trong đó (r, c) là tọa độ neo; $o \in \{\texttt{STANDING}, \texttt{HORIZONTAL}, \texttt{VERTICAL}\}$ là tư thế; và $\mathbf{b} = (b_1, \ldots, b_k)$ là vector Boolean biểu diễn trạng thái của k cầu. Trong cài đặt, $b_i = \texttt{True}$ nghĩa là cầu tương ứng đang mở và có thể đi qua.

Tập ô mà khối chiếm giữ được xác định bởi

$$\mathrm{Occ}(S) = \begin{cases} \{(r, c)\}, & o = \texttt{STANDING}, \\ \{(r, c), (r, c+1)\}, & o = \texttt{HORIZONTAL}, \\ \{(r, c), (r+1, c)\}, & o = \texttt{VERTICAL}. \end{cases} \tag{2.3}$$

2.2.2 Chế độ tách khối

Nếu đã kích hoạt split switch, trạng thái được mô tả bởi

$$S_{\text{split}} = (p_0, p_1, k, \mathbf{b}), \tag{2.4}$$

trong đó p_0, p_1 là tọa độ của hai khối đơn vị và $k \in \{0, 1\}$ là chỉ số khối đang được điều khiển. Trạng thái split phải giữ vị trí của cả hai khối; chỉ ghi tọa độ khối đang điều khiển sẽ làm mất thông tin cần thiết để xác định tính hợp lệ, khả năng nhập lại và những chuyển trạng thái tiếp theo.

Trong cài đặt, phép so sánh và hash trạng thái khối nguyên vẹn sử dụng $(r, c, o, \mathbf{b})$; còn trạng thái split sử dụng $(p_0, p_1, k, \mathbf{b})$. Do đó, cùng vị trí khối nhưng khác cấu hình cầu hoặc khác khối đang được điều khiển không bị gộp nhầm thành cùng một trạng thái.

2.3 Tập hành động, chuyển trạng thái và điều kiện đích

2.3.1 Tập hành động

Với khối nguyên vẹn, tập hành động được trả về bởi board là

$$\mathcal{A}_{\text{whole}} = \{U, D, L, R\}. \tag{2.5}$$

Khi tách khối, source bổ sung hành động `SPACE` để chuyển quyền điều khiển:

$$\mathcal{A}_{\text{split}} = \{U, D, L, R, \texttt{SPACE}\}. \tag{2.6}$$

Hành động `SPACE` chỉ có ý nghĩa trong chế độ split; nếu áp dụng trong chế độ khối nguyên vẹn, chuyển trạng thái không được chấp nhận.

2.3.2 Bảng chuyển động hình học

Bảng 2.1 thể hiện 12 trường hợp chuyển động của khối nguyên vẹn. Các công thức là chuyển động hình học trước bước kiểm tra ô, cập nhật công tắc và xác thực trạng thái. Một chuyển động hình học chỉ tạo thành cạnh hợp lệ của đồ thị nếu toàn bộ ô chiếm giữ mới hợp lệ theo cấu hình cầu tương ứng.

Bảng 2.1: Bảng chuyển động của khối nguyên vẹn

Tư thế hiện tại	Hành động	Neo mới	Tư thế mới	Các ô chiếm giữ mới
Đứng	U	$(r-2, c)$	VERTICAL	$(r-2, c), (r-1, c)$
	D	$(r+1, c)$	VERTICAL	$(r+1, c), (r+2, c)$
	L	$(r, c-2)$	HORIZONTAL	$(r, c-2), (r, c-1)$
	R	$(r, c+1)$	HORIZONTAL	$(r, c+1), (r, c+2)$
Nằm ngang	U	$(r-1, c)$	HORIZONTAL	$(r-1, c), (r-1, c+1)$
	D	$(r+1, c)$	HORIZONTAL	$(r+1, c), (r+1, c+1)$
	L	$(r, c-1)$	STANDING	$(r, c-1)$
	R	$(r, c+2)$	STANDING	$(r, c+2)$
Nằm dọc	U	$(r-1, c)$	STANDING	$(r-1, c)$
	D	$(r+2, c)$	STANDING	$(r+2, c)$
	L	$(r, c-1)$	VERTICAL	$(r, c-1), (r+1, c-1)$
	R	$(r, c+1)$	VERTICAL	$(r, c+1), (r+1, c+1)$

Đối với chế độ tách khối, các hành động di chuyển chỉ làm thay đổi tọa độ của khối đơn vị đang được điều khiển $p_k = (r, c)$, giữ nguyên vị trí khối còn lại p_{1-k}. Hành động `SPACE` chuyển quyền điều khiển mà không thay đổi tọa độ hình học.

Bảng 2.2: Bảng chuyển động trong chế độ tách khối (Split mode)

Hành động	Vị trí mới p'_k	Vị trí p'_{1-k}	Chỉ số k'	Cơ chế nhập lại
U	$(r-1, c)$	p_{1-k}	k	Nhập thành VERTICAL khi hai ô kề nhau theo phương dọc
D	$(r+1, c)$	p_{1-k}	k	Nhập thành VERTICAL khi hai ô kề nhau theo phương dọc
L	$(r, c-1)$	p_{1-k}	k	Nhập thành HORIZONTAL khi hai ô kề nhau theo phương ngang
R	$(r, c+1)$	p_{1-k}	k	Nhập thành HORIZONTAL khi hai ô kề nhau theo phương ngang
`SPACE`	p_k	p_{1-k}	$1-k$	Không

2.3.3 Điều kiện hợp lệ và tác dụng của công tắc

Hàm chuyển trạng thái của board trả về trạng thái kế tiếp nếu trạng thái đó hợp lệ; ngược lại trả về `None`. Một trạng thái bị loại khi tọa độ chiếm giữ vượt khỏi bàn, chạm ô vực, chạm cầu đang đóng, hoặc khối nguyên vẹn đứng trên ô fragile. Trong chế độ split, hai khối đơn vị không được trùng tọa độ; mỗi khối phải nằm trên ô hợp lệ.

Sau khi chuyển động hình học được kiểm tra, bộ máy xử lý cơ chế đặc biệt. Với khối nguyên vẹn, split switch chỉ tạo trạng thái Split khi khối vừa chuyển sang tư thế đứng thẳng tại ô switch và cấu hình có hai vị trí `split_positions`; tác động switch được áp dụng trước, sau đó các soft switch tại hai vị trí tách có thể tiếp tục cập nhật cầu. Với trạng thái Split, việc di chuyển và kiểm tra nhập lại được xử lý riêng. Các quy tắc cụ thể gồm:

- Soft switch được kích hoạt nếu ô công tắc thuộc phần chiếm giữ mới của khối nguyên vẹn, bất kể tư thế; khối đơn vị cũng kích hoạt được soft switch.
- Heavy switch chỉ được kích hoạt khi khối nguyên vẹn đứng thẳng tại ô công tắc.
- Split switch chỉ được xử lý khi khối nguyên vẹn chuyển sang tư thế đứng thẳng tại ô đó; hai khối đơn vị được đặt theo cấu hình `split_positions`.
- Với chế độ split, di chuyển một khối đơn vị có thể làm hai khối nhập lại nếu chúng ở hai ô kề cạnh theo phương ngang hoặc phương dọc.
- Trạng thái bridge được cập nhật theo tác động của switch và được đưa vào trạng thái kế tiếp. Vì vậy, cùng một vị trí hình học với cấu hình bridge khác nhau là các trạng thái khác nhau.

2.3.4 Trạng thái ban đầu và kiểm tra đích

Trạng thái đầu S_0 được tạo từ tọa độ `start` trong JSON, với tư thế `STANDING` và các cờ cầu lấy từ `initial_state`. Hàm kiểm tra đích chấp nhận trạng thái khi và chỉ khi khối chưa bị tách, đang đứng thẳng, và tọa độ neo trùng với tọa độ `goal`:

$$\text{Goal}(S) = \text{True} \iff \neg\,\text{Split}(S) \wedge o = \texttt{STANDING} \wedge (r, c) = (r_g, c_g). \tag{2.7}$$

2.4 Breadth-First Search (BFS)

BFS dùng hàng đợi FIFO và mở rộng trạng thái theo thứ tự không giảm của số hành động từ trạng thái đầu. Trạng thái được đánh dấu đã phát hiện ngay khi đưa vào hàng đợi; một ánh xạ `parent` lưu cặp trạng thái cha và hành động để dựng lại đường đi khi tìm thấy đích.

Listing 2.1: Pseudocode của BFS

```
BFS(S0):
    queue <- FIFO([S0]); visited <- {S0}; parent <- empty map
    while queue is not empty:
        S <- pop_front(queue)
        if Goal(S): return reconstruct_path(parent, S)
        for a in Actions(S):
            S2 <- T(S, a)
            if S2 is valid and S2 not in visited:
                visited.add(S2)
                parent[S2] <- (S, a)
                push_back(queue, S2)
    return failure
```

Tính chất. Trên đồ thị trạng thái hữu hạn, BFS đầy đủ nếu mọi trạng thái kế tiếp hợp lệ được sinh đầy đủ. Khi mọi hành động có chi phí đồng nhất bằng 1, BFS trả về nghiệm có số hành động nhỏ nhất. BFS không tối ưu tổng chi phí của một hàm chi phí phi đồng nhất.

2.5 Depth-First Search (DFS)

DFS sử dụng ngăn xếp LIFO để ưu tiên đi sâu theo một nhánh trước khi quay lui. Cài đặt dùng ánh xạ `parent` đồng thời làm dấu nhận biết trạng thái đã xử lý; các bản ghi trùng bị bỏ qua khi lấy khỏi ngăn xếp. Thứ tự hành động được đảo khi đẩy vào ngăn xếp để thứ tự duyệt khi lấy ra phù hợp với danh sách hành động.

DFS có thể tìm được nghiệm trên đồ thị hữu hạn nếu duyệt đúng các cạnh và dùng cơ chế phát hiện trạng thái lặp, nhưng không đảm bảo nghiệm có số bước nhỏ nhất. Độ dài nghiệm phụ thuộc vào thứ tự hành động và cấu trúc bàn chơi. Do phiên bản này lưu cha cho các trạng thái đã xử lý, bộ nhớ có thể tăng theo số trạng thái được ghi nhận, không phải chỉ theo độ sâu của một nhánh.

Listing 2.2: Pseudocode của DFS theo implementation

```
DFS(S0):
    stack <- [(S0, no_parent, no_action)]; parent <- empty map
    while stack is not empty:
        (S, P, a_in) <- pop(stack)
        if S in parent: continue
        parent[S] <- (P, a_in)
        if Goal(S): return reconstruct_path(parent, S)
        for a in reverse(Actions(S)):
            S2 <- T(S, a)
            if S2 is valid and S2 not in parent:
                push(stack, (S2, S, a))
    return failure
```

2.6 Iterative Deepening Search (IDS)

IDS tăng giới hạn độ sâu $L = 0, 1, 2, \ldots$ và ở mỗi lượt thực hiện tìm kiếm DFS có giới hạn. Trong từng lượt, cài đặt sử dụng `best_depth` để lưu độ sâu tốt nhất đã biết của mỗi trạng thái; trạng thái được xem xét lại khi tìm được một đường đi đến nó ngắn hơn đường đã ghi nhận. Bộ đếm số trạng thái đã chạm tới giữa hai lượt được dùng làm điều kiện dừng khi không còn mở rộng thêm không gian trạng thái.

Với chi phí mỗi hành động bằng 1, IDS hướng tới nghiệm có số hành động nhỏ nhất. Trong implementation, mỗi lượt dùng `best_depth` để cho phép xem xét lại một trạng thái nếu tìm được đường đi ngắn hơn; số trạng thái được ghi nhận được dùng làm điều kiện dừng khi không tăng sau khi mở rộng giới hạn. Điều kiện này dựa trên giả định rằng không gian trạng thái hữu hạn và mỗi lượt duyệt đầy đủ các trạng thái đạt được trong giới hạn tương ứng. Do đó, tính đầy đủ của điều kiện dừng cần được xác nhận cùng mô hình chuyển trạng thái và cách loại trạng thái lặp. Đây không phải IDS cây tìm kiếm cổ điển: code lưu `best_depth` và `parent` trong từng lượt, nên không nên gán trực tiếp cận bộ nhớ $O(bd)$ cho implementation này.

Listing 2.3: Pseudocode của IDS theo implementation

```
IDS(S0):
    limit <- 0; previous_count <- -1
    loop:
        (path, visited_count) <- depth_limited_search(S0, limit)
        if path is not failure: return path
        if visited_count == previous_count: return failure
        previous_count <- visited_count
        limit <- limit + 1

depth_limited_search(S0, limit):
    stack <- [(S0, 0, no_parent, no_action)]
    best_depth <- empty map; parent <- empty map
    while stack is not empty:
        (S, depth, P, a_in) <- pop(stack)
        if S in best_depth and best_depth[S] <= depth: continue
        best_depth[S] <- depth; parent[S] <- (P, a_in)
        if Goal(S): return (reconstruct_path(parent, S), size(best_depth))
        if depth == limit: continue
        for a in reverse(Actions(S)):
            S2 <- T(S, a)
            if S2 is valid and not reached at depth <= depth+1:
                push(stack, (S2, depth+1, S, a))
    return (failure, size(best_depth))
```

2.7 Uniform-Cost Search (UCS)

Uniform-Cost Search (UCS) luôn mở rộng nút có tổng chi phí đường đi đã biết nhỏ nhất $g(S)$. Khác với BFS chỉ tối ưu số cạnh khi mọi cạnh có cùng chi phí, UCS cho phép mỗi chuyển trạng thái có một chi phí không âm được xác định bởi hàm $c(S, a, S')$. Trong implementation, một min-heap lưu các nút biên, còn `best_g` ghi nhận chi phí tốt nhất tìm được đến mỗi nút. Các mục heap lỗi thời được bỏ qua khi chi phí lấy ra lớn hơn giá trị hiện hành trong `best_g`.

Với mô hình chi phí được chọn cho thực nghiệm, chi phí của một hành động lăn có thể phụ thuộc vào hành động lăn ngay trước đó. Do đó, chỉ dùng trạng thái vật lý S làm khóa tìm kiếm có thể gộp hai đường đi có chi phí tương lai khác nhau. Khi hàm chi phí khai báo `depends_on_last_action=True`, implementation mở rộng khóa thành

$$N = (S, \ell), \tag{2.8}$$

trong đó ℓ là hành động lăn gần nhất thuộc $\{U, D, L, R\}$; giá trị này được đặt lại thành rỗng sau hành động chuyển quyền điều khiển như `SPACE`. Cách biểu diễn này làm cho thông tin cần thiết để tính chi phí chuyển tiếp trở thành một phần của nút tìm kiếm.

Listing 2.4: Pseudocode của UCS với mô hình chi phí phụ thuộc hành động trước

```
UCS(S0):
    start <- (S0, NONE)
    best_g[start] <- 0
    pq <- min_heap([(0, next_counter(), start)])
    parent <- empty map

    while pq is not empty:
        (g, _, node) <- pop_min(pq)
        if g > best_g[node]: continue       # stale heap entry
        (S, last_roll) <- node

        if Goal(S):
            return reconstruct_path(parent, node), g

        for a in Actions(S):
            S2 <- T(S, a)
            if S2 is invalid: continue

            step_cost <- C(S, last_roll, a, S2)
            last2 <- a if a in {U,D,L,R} else NONE
            node2 <- (S2, last2)
            g2 <- g + step_cost

            if g2 < best_g.get(node2, infinity):
                best_g[node2] <- g2
                parent[node2] <- (node, a)
                push(pq, (g2, next_counter(), node2))

    return failure
```

Với cạnh có chi phí không âm, UCS tìm được đường đi có tổng chi phí nhỏ nhất trong đồ thị trạng thái hữu hạn, miễn là trạng thái, khóa tìm kiếm và các chuyển tiếp được mô hình hóa đầy đủ. Do một số hành động điều khiển có chi phí bằng 0 trong mô hình được chọn, điều kiện cần dùng ở đây là chi phí *không âm*, không phải mọi cạnh đều có chi phí dương.

2.7.1 Các mô hình chi phí có trong mã nguồn

Mã nguồn có một số cost model để đối chiếu. Lớp `UniformCost` trả về chi phí 1 cho mỗi hành động. `PhysicsBasedCost` bắt đầu từ chi phí 1 và cộng 0.5 khi chuyển từ tư thế nằm sang đứng:

$$c_{\text{physics}}(S, a, S') = \begin{cases} 1.5, & \neg\,\text{Standing}(S) \wedge \text{Standing}(S'), \\ 1.0, & \text{các trường hợp còn lại.} \end{cases} \tag{2.9}$$

Mô hình này biểu diễn một giả định chi phí mang tính trực giác, không phải phép đo năng lượng cơ học đã được hiệu chuẩn. Lớp `RiskSafetyCost` cộng chi phí cho các ô fragile và các ô gần vực theo phép kiểm tra trong code. Riêng phép kiểm tra mép vực của lớp này dựa trên lưới tĩnh và biên bàn, nên không mô tả đầy đủ mọi rủi ro động do trạng thái cầu.

Cần phân biệt các lớp trên với baseline chi phí không có khoản phạt trong thí nghiệm tuning. Baseline đó gán chi phí 1 cho mỗi lần lăn U, D, L, R và chi phí 0 cho thao tác điều khiển `SPACE`; nó không hoàn toàn đồng nhất với lớp `UniformCost`, vốn trả về 1 cho cả `SPACE`. Sự phân biệt này phù hợp với quy ước của thực nghiệm rằng `SPACE` là thao tác đổi khối điều khiển, không phải lần lăn vật lý.

2.7.2 Hàm chi phí tổng hợp được dùng trong thực nghiệm

Solver UCS và A* sử dụng chung lớp `TunedWeightCost`. Với hành động lăn $a \in \{U, D, L, R\}$, chi phí được tính theo

$$\begin{aligned} c_{\boldsymbol{w}}(S, \ell, a, S') = w_{\text{step}} &+ w_{\text{back}} I_{\text{back}}(\ell, a) + w_{\text{void}} I_{\text{void}}(S') \\ &+ w_{\text{frag}} M_{\text{frag}}(S, S') + w_{\text{switch}} M_{\text{switch}}(S, S') \\ &+ w_{\text{bridge}} M_{\text{bridge}}(S, S'), \end{aligned} \tag{2.10}$$

trong khi hành động điều khiển như `SPACE` có chi phí bằng 0. Các thành phần được xác định trực tiếp từ chuyển tiếp:

- $I_{\text{back}} = 1$ nếu hành động lăn hiện tại đảo chiều ngay lập tức hành động lăn trước đó; nếu không thì bằng 0. Thao tác đổi khối điều khiển đặt lại ngữ cảnh đảo chiều.
- $I_{\text{void}} = 1$ nếu trạng thái sau chuyển tiếp có ít nhất một ô đang chiếm giữ với từ hai hướng kề không thể đi vào trở lên (do ngoài biên, ô không hợp lệ hoặc cấu hình cầu); khoản phạt này chỉ cộng một lần cho chuyển tiếp.
- M_{frag} là số ô fragile mới được khối chiếm giữ trong chuyển tiếp; đây là khoản phạt tiếp xúc/đi vào ô fragile, không phải mô hình hóa một lần di chuyển thất bại.
- M_{switch} là số cờ trạng thái cầu thay đổi trong chuyển tiếp do cơ chế của board.
- M_{bridge} là số ô cầu mới được khối đi vào.

Trọng số bước w_{step} được cố định ở 1.0 để xác định thang đo chi phí. Năm trọng số phạt được khảo sát trên lưới rời rạc:

$$\begin{aligned} &w_{\text{back}} \in \{0, 3, 5\}, \quad w_{\text{void}} \in \{0, 2, 4\}, \\ &w_{\text{frag}} \in \{0, 4, 8\}, \quad w_{\text{switch}} \in \{0, 2.5, 5\}, \quad w_{\text{bridge}} \in \{0, 3, 5\}. \end{aligned} \tag{2.11}$$

Lưới gồm $3^5 = 243$ tổ hợp và được đối chiếu thêm với baseline đồng trọng số. Quy trình chọn

mô hình dùng sáu fold theo nhóm level, mỗi fold giữ lại ba level nguyên vẹn để đánh giá. Hàm mất mát đa tiêu chí dựa trên thứ hạng chuẩn hóa trong từng level, xét số lần lăn, đảo chiều, phơi nhiễm mép vực, đi vào ô fragile, thay đổi trạng thái cầu và đi vào ô cầu; mức ưu tiên được quy định trước theo tỷ lệ $(1, 3, 2, 4, 2.5, 3)$. Lời giải thất bại hoặc không hợp lệ nhận mức phạt lớn hơn các lời giải hợp lệ.

Kết quả cross-validation không cho thấy cấu hình tìm kiếm thay thế cải thiện ổn định trên các level giữ lại. Vì vậy, cấu hình cuối cùng được giữ ở bộ trọng số rubric ban đầu:

$$\boxed{(w_{\text{back}}, w_{\text{void}}, w_{\text{frag}}, w_{\text{switch}}, w_{\text{bridge}}) = (3, 2, 4, 2.5, 3).} \tag{2.12}$$

Đây là lựa chọn được thử nghiệm dựa trên kết quả grouped cross-validation, không khẳng định rằng vector này là tối ưu toàn cục. Kết quả định lượng và các baseline so sánh được trình bày trong chương thực nghiệm. Các trọng số được xác định bằng tìm kiếm lưới trên tiêu chí kỹ thuật do nhóm định nghĩa; không có bước học có giám sát từ nhãn đánh giá độc lập, vì vậy nhóm không gọi đây là các trọng số đã được mô hình học tự động.

2.8 A* Search

A* là thuật toán tìm kiếm có thông tin, kết hợp chi phí đường đi đã đi với một hàm heuristic ước lượng chi phí còn lại. Hàm ưu tiên được viết dưới dạng

$$f(N) = g(N) + h_\alpha(S), \qquad h_\alpha(S) = \alpha h_0(S), \tag{2.13}$$

trong đó N là nút tìm kiếm, S là trạng thái Bloxorz chứa trong nút, $g(N)$ là tổng chi phí thực tế từ trạng thái đầu đến nút hiện tại, $h_0(S)$ là heuristic nền và α là hệ số tỉ lệ. Trong implementation đang được đánh giá, A* dùng cùng mô hình chi phí `TunedWeightCost` với UCS; do đó g là tổng chi phí có trọng số, không nhất thiết bằng số lần lăn hoặc độ dài danh sách hành động.

Vì chi phí của `TunedWeightCost` có thể phụ thuộc vào việc hành động lăn hiện tại có đảo chiều hành động lăn trước hay không, khi bật cost model này khóa tìm kiếm gồm (S, ℓ) như trong Mục 2.7. Heuristic chỉ phụ thuộc trạng thái S, không phụ thuộc ngữ cảnh ℓ. A* lưu các nút trên min-heap, cập nhật `g_score` khi tìm được đường đi có chi phí thấp hơn, và bỏ qua các mục heap lỗi thời.

2.8.1 Heuristic nền được triển khai

Với khối nguyên vẹn, đặt $\text{Occ}(S)$ là tập ô đang bị chiếm giữ và (r_g, c_g) là vị trí đích. Khoảng cách Manhattan gần nhất từ các ô này tới đích là

$$d_{\text{whole}}(S) = \min_{(r,c)\in \text{Occ}(S)} \left(|r - r_g| + |c - c_g|\right). \tag{2.14}$$

Heuristic nền của khối nguyên vẹn được tính bởi

$$h_0(S) = \frac{d_{\text{whole}}(S)}{2} + \begin{cases} 0, & \text{nếu khối đang đứng thẳng,} \\ 0.5, & \text{nếu khối đang nằm.} \end{cases} \tag{2.15}$$

Khoảng cách được chia cho 2 vì một lần lăn có thể làm thay đổi vị trí các ô chiếm giữ tối đa hai đơn vị theo một trục. Khoản hiệu chỉnh 0.5 phản ánh rằng khối nằm chưa thỏa điều kiện đích là đứng thẳng.

Trong chế độ Split, với hai khối đơn vị tại $p_0 = (r_0, c_0)$ và $p_1 = (r_1, c_1)$, đặt

$$d_{\text{inter}}(S) = |r_0 - r_1| + |c_0 - c_1|, \tag{2.16}$$

$$d_0(S) = |r_0 - r_g| + |c_0 - c_g|, \tag{2.17}$$

$$d_1(S) = |r_1 - r_g| + |c_1 - c_g|. \tag{2.18}$$

Implementation sử dụng công thức

$$h_0(S) = \max\left(0, \frac{d_{\text{inter}}(S) - 1}{2}\right) + \frac{\min(d_0(S), d_1(S))}{2} + 1.0. \tag{2.19}$$

Số hạng đầu định hướng hai khối tiến lại gần nhau để có khả năng hợp nhất; số hạng thứ hai định hướng ít nhất một khối về phía ô đích; hằng số 1.0 là hiệu chỉnh thực nghiệm cho trạng thái Split. Công thức này được nêu đúng theo implementation, nhưng không được mặc nhiên coi là một cận dưới đã được chứng minh cho mọi trạng thái Split.

2.8.2 Tinh chỉnh hệ số heuristic

Trong thí nghiệm, hàm chi phí C của A* được giữ đồng nhất với cấu hình UCS đã chọn, còn hệ số α được khảo sát trên tập

$$\alpha \in \{0, 0.25, 0.5, 0.75, 1.0\}. \tag{2.20}$$

Giá trị $\alpha = 0$ tương ứng với việc không dùng heuristic và ưu tiên theo g; giá trị $\alpha = 1$ giữ nguyên heuristic nền của implementation. Ở mỗi fold, việc lựa chọn ưu tiên các giá trị vẫn cho lời giải hợp lệ và có tổng chi phí khớp với UCS, sau đó chọn giá trị có số trạng thái mở rộng trung bình thấp nhất; thời gian tìm kiếm được dùng làm chỉ số phụ vì dao động theo môi trường chạy.

Cả sáu fold grouped cross-validation đều chọn $\alpha = 1.0$, và cấu hình này cũng được chọn khi đánh giá trên toàn bộ 18 level. Trong đợt dò hệ số, số nút mở rộng trung bình giảm từ khoảng 1285.8 tại $\alpha = 0$ xuống 989.7 tại $\alpha = 1.0$ (giảm xấp xỉ 23.0%). Do đó, thí nghiệm có khảo sát hệ số heuristic, nhưng cấu hình cuối cùng không thay đổi công thức heuristic nền: chỉ số tỉ lệ được chọn vẫn là 1.0. Chi tiết so sánh được trình bày ở chương thực nghiệm.

2.8.3 Tính chất và giới hạn của heuristic

Trên các bản đồ chỉ có cơ chế khối nguyên vẹn, công thức (2.15) có thể được phân tích với chi phí cơ sở 1 cho mỗi lần lăn. Khoảng cách Manhattan gần nhất giảm tối đa hai đơn vị qua một lần lăn; khi chuyển giữa tư thế đứng và nằm, khoản hiệu chỉnh tư thế bù cho giới hạn biến thiên này. Vì vậy, với mô hình chuyển động khối nguyên vẹn, heuristic nền là admissible và consistent. Do các khoản phạt của C không âm và mỗi lần lăn có chi phí ít nhất bằng 1, lập luận cận dưới này vẫn phù hợp với các cạnh lăn trên những bản đồ không có cơ chế Split.

Đối với chế độ Split, không nên mở rộng kết luận lý thuyết trên một cách tự động. Kiểm tra đồ thị trạng thái dùng chi phí cơ sở bằng 1 cho các lần lăn và 0 cho thao tác đổi khối điều khiển cho thấy không có vi phạm admissibility hoặc consistency ở các level 1–14. Trên các level Split 15–18, bộ kiểm tra ghi nhận các vi phạm của heuristic nền: lần lượt có 6, 34, 16 và 20 trạng

thái vi phạm admissibility, cùng 38, 402, 322 và 348 cạnh vi phạm consistency. Đây là kết quả kiểm tra với chi phí chuyển động cơ sở; nó không tự nó chứng minh mức admissibility dưới toàn bộ hàm chi phí có trọng số, vì các khoản phạt bổ sung có thể làm thay đổi chi phí tối ưu còn lại.

Trong benchmark thực nghiệm, mọi giá trị α được thử đều trả về lời giải hợp lệ với tổng chi phí khớp UCS trên cả 18 level. Đây là bằng chứng thực nghiệm cho tập benchmark hiện có, không thay thế một chứng minh tối ưu tổng quát khi heuristic Split chưa được chứng minh admissible và consistent dưới đúng hàm chi phí C. Vì vậy, trong báo cáo cần phân biệt rõ tính chất lý thuyết đã chứng minh cho bản đồ không Split với kết quả quan sát trên các level có Split.

2.8.4 Pseudocode của A*

Pseudocode dưới đây tóm tắt implementation A* sử dụng mô hình chi phí có thể phụ thuộc hành động lăn trước. Nút lưu trong heap chứa trạng thái, ngữ cảnh hành động trước và đường đi tương ứng.

Listing 2.5: Pseudocode của A* với hàm chi phí có trọng số

```
AStar(S0):
    start <- (S0, NONE)
    g_score[start] <- 0
    pq <- min_heap(
        [(alpha*h0(S0), 0, next_counter(), start, [])])

    while pq is not empty:
        (f, g, _, node, path) <- pop_min(pq)
        if g > g_score[node]: continue
        (S, last_roll) <- node

        if Goal(S): return path, g

        for a in Actions(S):
            S2 <- T(S, a)
            if S2 is invalid: continue

            g2 <- g + C(S, last_roll, a, S2)
            last2 <- a if a in {U,D,L,R} else NONE
            node2 <- (S2, last2)

            if g2 < g_score.get(node2, infinity):
                g_score[node2] <- g2
                f2 <- g2 + alpha*h0(S2)
                push(pq, (f2, g2, next_counter(),
                          node2, path + [a]))

    return failure
```

Các quy tắc về tính hợp lệ của ô, tác động của công tắc và cập nhật trạng thái cầu được xử lý trong hàm chuyển trạng thái $T(S, a)$. Implementation lưu trực tiếp danh sách `path` trong từng mục heap; cách làm này đơn giản hóa việc trả về lời giải nhưng có thể sao chép danh sách nhiều lần và làm tăng chi phí bộ nhớ thực tế.

2.9 Greedy Best-First Search (GBFS) – phương pháp bổ sung

Greedy Best-First Search (GBFS) ưu tiên mở rộng trạng thái có giá trị heuristic $h(S)$ nhỏ nhất. Không giống UCS và A*, GBFS không cộng chi phí đường đi đã đi $g(S)$ vào khóa ưu tiên. Vì vậy, nó có thể mở rộng ít trạng thái và tìm lời giải nhanh trên một số level, nhưng không bảo đảm tìm được đường đi có số lần lăn ít nhất hoặc có tổng chi phí theo hàm C nhỏ nhất.

Implementation duy trì frontier bằng min-heap; mỗi phần tử gồm giá trị heuristic, bộ đếm phụ, trạng thái và danh sách hành động tương ứng. Tập `visited` tránh phát hiện lại trạng thái đã gặp. GBFS dùng heuristic riêng và không sử dụng hàm chi phí tổng hợp C của UCS/A* để sắp xếp frontier.

2.9.1 Heuristic và các tham số

Với trạng thái khối nguyên vẹn, implementation sử dụng tọa độ neo (r, c) của trạng thái; với trạng thái Split, nó dùng vị trí của cả hai khối. Đặt hàm khoảng cách nền là

$$D(S) = \begin{cases} \dfrac{|r - r_g| + |c - c_g|}{2}, & \text{khối nguyên vẹn,} \\ \max\left(0, \dfrac{d_{\text{inter}}(S) - 1}{2}\right) + \dfrac{\min(d_0(S), d_1(S))}{2}, & \text{trạng thái Split,} \end{cases} \tag{2.21}$$

trong đó d_{inter}, d_0 và d_1 được định nghĩa như trong phần heuristic Split của A*. Các thành phần hiệu chỉnh còn lại là:

$$O(S) = \begin{cases} 1, & \text{khối nguyên vẹn không đứng thẳng,} \\ 0, & \text{ngược lại,} \end{cases} \tag{2.22}$$

$$P(S) = \begin{cases} 1, & \text{trạng thái Split,} \\ 0, & \text{khối nguyên vẹn,} \end{cases} \tag{2.23}$$

$$E(S) = \text{số ô đang chiếm giữ được xác định là gần vực,} \tag{2.24}$$

$$F(S) = \text{số ô fragile đang bị chiếm giữ.} \tag{2.25}$$

Heuristic dùng trong GBFS là

$$h_{\boldsymbol{\theta}}(S) = D(S) + w_{\text{orientation}} O(S) + w_{\text{split}} P(S) + w_{\text{edge}} E(S) + w_{\text{fragile}} F(S). \tag{2.26}$$

Ở đây, w_{edge} phạt số ô đang chiếm giữ được xác định là gần vực theo hàm kiểm tra trong code; w_{fragile} đếm số ô fragile hiện đang bị chiếm giữ, không phải số ô mới đi vào ở bước chuyển tiếp. Các thành phần địa hình này chỉ ảnh hưởng thứ tự ưu tiên của GBFS, không phải chi phí tích lũy của đường đi.

Nhóm đã khảo sát bốn tham số trên lưới rời rạc gồm

$$w_{\text{orientation}} \in \{0, 0.5, 1, 1.5\}, \qquad w_{\text{split}} \in \{0, 1, 2, 3\},$$
$$w_{\text{edge}} \in \{0, 0.5, 1, 2\}, \qquad w_{\text{fragile}} \in \{0, 1, 2, 4\}.$$

Tổng cộng có $4^4 = 256$ cấu hình heuristic được đánh giá trên 18 level. Việc chọn tham số dựa trên cùng tiêu chí mất mát đa tiêu chí theo thứ hạng và cùng cách chia grouped cross-validation

theo level như thí nghiệm UCS. Cấu hình có điểm tốt nhất khi đánh giá trên toàn bộ dữ liệu không được tự động chọn nếu không cải thiện kết quả trên các level giữ lại.

Kết quả cross-validation không cho thấy một cấu hình mới cải thiện ổn định so với heuristic ban đầu. Do đó, nhóm giữ cấu hình

$$\boxed{(w_{\text{orientation}}, w_{\text{split}}, w_{\text{edge}}, w_{\text{fragile}}) = (1.0, 1.0, 0.0, 0.0).} \tag{2.27}$$

Cấu hình này vẫn cộng hiệu chỉnh 1.0 cho trạng thái nằm hoặc Split, nhưng không thêm phạt mép vực hay ô fragile. Vì vậy, dẫu cho nhóm đã thực hiện grid search và cross-validation cho heuristic GBFS nhưng giữ lại cấu hình ban đầu do chưa có cải thiện trên tập giữ lại;

2.9.2 Pseudocode

Listing 2.6: Pseudocode của GBFS theo implementation

```
GBFS(S0):
    if Goal(S0): return []
    counter <- 0
    frontier <- min_heap()
    push(frontier, (h_theta(S0), counter, S0, []))
    visited <- {S0}

    while frontier is not empty:
        (_, _, S, path) <- pop_min(frontier)
        expanded_nodes <- expanded_nodes + 1
        if Goal(S): return path

        for a in Actions(S):
            S2 <- T(S, a)
            if S2 is not invalid and S2 not in visited:
                visited.add(S2)
                counter <- counter + 1
                push(frontier, (h_theta(S2), counter,
                                S2, path + [a]))
    return failure
```

2.9.3 Giới hạn và vai trò trong thực nghiệm

Heuristic GBFS là công cụ định hướng thứ tự mở rộng, không phải cận dưới được chứng minh của chi phí tối ưu còn lại. GBFS có thể tìm lời giải hợp lệ rất nhanh nhưng lời giải đó có thể có nhiều lần lăn hơn, nhiều thao tác chuyển quyền điều khiển hơn hoặc có đặc trưng địa hình kém hơn kết quả của UCS/A*. Vì vậy, khi đánh giá GBFS, nhóm báo cáo riêng tính hợp lệ của lời giải, số lần lăn, tổng số hành động, số trạng thái mở rộng, thời gian và bộ nhớ; không so sánh "tổng chi phí C" như là GBFS đã tối ưu theo hàm chi phí này.

2.10 Tính chất và độ phức tạp

Gọi V là số trạng thái có thể tiếp cận, E là số chuyển trạng thái hợp lệ, b là hệ số phân nhánh và d là độ sâu của nghiệm nông nhất. Bảng 2.3 tóm tắt các tính chất lý thuyết và lưu ý cài đặt thực tế của các thuật toán. Các cận lý thuyết áp dụng cho mô hình tìm kiếm trên đồ thị chuẩn; chi phí sao chép danh sách đường đi và quy ước đếm node thực tế có ảnh hưởng trực tiếp đến số liệu thực nghiệm.

2.11 Quy ước đếm node và đo đạc trong chương trình

Lớp `BaseSolver.solve()` đo thời gian bằng hàm `perf_counter()` của mô-đun `time` và đo đỉnh cấp phát Python bằng `tracemalloc`. Vùng đo bao quanh lời gọi `_search()`; do đó, thời gian nạp tệp JSON và khởi tạo `Board` nằm ngoài vùng đo nếu bàn chơi được tạo trước khi gọi solver.

Thuộc tính `solution_length` trong `BaseSolver` là độ dài danh sách action trả về, vì vậy có tính cả `SPACE` nếu action này xuất hiện. Nếu bảng thực nghiệm báo cáo số lần lăn khối, cần tính riêng số action thuộc $\{U, D, L, R\}$ và nêu rõ quy ước. Thuộc tính `expanded_nodes` được cập nhật riêng trong từng thuật toán tìm kiếm. Cụ thể: IDS cộng dồn số trạng thái được xử lý qua nhiều lượt giới hạn độ sâu; các mục heap lỗi thời của UCS và A* bị bỏ qua trước khi tăng bộ đếm. Đây là chỉ số đo đạc theo quy ước triển khai mã nguồn, do đó phần thực nghiệm bắt buộc phải sử dụng cùng một phiên bản source code và giải thích nhất quán quy ước này trong báo cáo.

Bảng 2.3: Tính chất lý thuyết và lưu ý triển khai

Thuật toán	**Đầy đủ và tối ưu**	**Độ phức tạp / lưu ý triển khai**
BFS	Đầy đủ trên đồ thị hữu hạn nếu sinh đầy đủ cạnh hợp lệ. Tối ưu theo số action khi mọi action có cost bằng 1.	Với đồ thị tường minh: $O(V+E)$ thời gian và $O(V)$ bộ nhớ phụ trợ cho `queue`, `visited` và `parent`.
DFS	Đầy đủ trên đồ thị hữu hạn khi duyệt đầy đủ và tránh trạng thái lặp. Không bảo đảm nghiệm ngắn nhất.	Bản cài đặt lưu `parent`/`visited` theo trạng thái; cận thực tế là $O(V+E)$ thời gian và tối đa $O(V)$ bộ nhớ.
IDS	Tối ưu theo số action khi cost đồng nhất và quá trình tăng giới hạn độ sâu được thực hiện đúng.	Bản này lặp các lượt giới hạn độ sâu và lưu `best_depth`/`parent` trong từng lượt; cận thận trọng là $O(d(V+E))$ thời gian và $O(V+E)$ bộ nhớ cho cấu trúc trạng thái/frontier.
UCS	Tối ưu theo tổng chi phí không âm với điều kiện khóa trạng thái giữ đủ ngữ cảnh ảnh hưởng đến chi phí.	Với `TunedWeightCost`, khóa gồm (S, ℓ) do phạt đảo chiều phụ thuộc hành động lăn trước; thao tác `SPACE` có chi phí 0. Trọng số cuối cùng $(3, 2, 4, 2.5, 3)$ được giữ lại sau grouped cross-validation.
A*	Sử dụng cùng hàm chi phí có trọng số với UCS và heuristic $h_\alpha = \alpha h_0$; $\alpha = 1.0$ được chọn trong sáu fold. Tính chất của heuristic Split cần được hiểu theo các giới hạn ở Mục 2.8.	Khóa tìm kiếm gồm (S, ℓ) cho hàm chi phí phụ thuộc hành động trước; lưu trực tiếp danh sách `path` trong heap. Kết quả khớp chi phí UCS trên 18 level là bằng chứng thực nghiệm, không phải chứng minh tối ưu tổng quát trên mọi trạng thái Split.
GBFS	Dùng heuristic có trọng số để sắp xếp frontier; không bảo đảm tối ưu theo số lần lăn hoặc hàm chi phí C. Cấu hình cuối cùng $(1, 1, 0, 0)$ được giữ lại vì grouped cross-validation không cho thấy cấu hình mới tốt hơn.	Không sử dụng $g(S)$ hoặc hàm chi phí C trong ưu tiên; các trọng số hiệu chỉnh heuristic được grid search trên 256 cấu hình. Mỗi trạng thái được đánh dấu visited khi phát hiện; đường đi lưu trực tiếp trong heap.

CHƯƠNG 3

Thiết lập thực nghiệm và đánh giá

3.1 Mục tiêu và nguyên tắc thực nghiệm

Thực nghiệm đánh giá các thuật toán trên cùng một bộ 18 level JSON, cùng mô hình trạng thái và cùng hàm chuyển trạng thái của lớp `Board`. Mỗi solver được khởi tạo mới cho từng lượt chạy. Mọi nghiệm được tìm thấy đều được phát lại từ trạng thái đầu; một lượt chỉ được ghi nhận thành công khi mọi hành động đều tạo trạng thái hợp lệ và trạng thái cuối thỏa `board.is_goal`.

Cần phân biệt hai mục tiêu. Thứ nhất là kiểm thử chức năng: xác minh solver có thể giải toàn bộ 18 level với đầy đủ các cơ chế từ cơ bản đến phức tạp (ô mỏng, công tắc cầu và phân tách khối). Thứ hai là benchmark hiệu năng: so sánh thời gian, bộ nhớ, số node và chất lượng đường đi qua 540 lượt thử nghiệm (5 lượt lặp cho mỗi cặp level và solver). Bộ 18 level cung cấp không gian trạng thái đủ lớn và nhiều tuyến đường cạnh tranh, cho phép đánh giá thuyết phục tính tối ưu bước đi cũng như hành vi an toàn của các thuật toán.

3.2 Môi trường và quy trình chạy

Bảng 3.1 mô tả chi tiết môi trường phần cứng, hệ điều hành và quy trình đo lường được sử dụng để tiến hành toàn bộ các đợt thực nghiệm benchmark trong báo cáo này. Việc thực hiện toàn bộ đo đạc trên cùng một thiết bị giúp đảm bảo tính nhất quán của số liệu về thời gian thực thi và dung lượng bộ nhớ.

Bảng 3.1: Môi trường thực thi và thông số máy kiểm nghiệm benchmark

Thông số	**Cấu hình ghi nhận**	**Quy ước và Ghi chú thực thi**
Hệ điều hành	Microsoft Windows 11 Home	64-bit, Build 26200; thực thi trực tiếp trên hệ thống Native Windows.
CPU	13th Gen Intel(R) Core(TM) i5-13420H	Kiến trúc 8 nhân (4 P-Cores + 4 E-Cores), 12 luồng logic, xung nhịp tối đa 4.60 GHz.
Bộ nhớ RAM	16.0 GB	15.71 GiB bộ nhớ vật lý khả dụng; đảm bảo tài nguyên dồi dào cho các thuật toán tìm kiếm không gian lớn.
Python Runtime	Python 3.11.9 (64-bit)	Bản dựng chính thức MSC v.1938 AMD64.
Bộ test thực nghiệm	18 màn chơi JSON (`level_01` → `level_18`)	Đầy đủ các cơ chế: nút tròn, nút chữ X, ô cam sập sàn, cầu nối liên kết và công tắc tách khối (*split switch*).
Cấu hình Solver	6 thuật toán tìm kiếm	Gồm `BFS`, `UCS`, `DFS`, `IDS`, `A*`, và `Greedy`. Tổng cộng thực hiện $18 \times 6 \times 5 = 540$ lượt thực nghiệm (mỗi cặp lặp 5 lần để triệt tiêu sai số).
Quy trình đo đạc	Thực thi tự động qua `benchmark.py`	Gọi trực tiếp API `solve()` nguyên bản; đo thời gian bằng `time.perf_counter()` (ms), đo RAM đỉnh bằng `tracemalloc` (KiB), thiết lập timeout an toàn 15.0s mỗi lượt.

3.3 Mô tả Chi tiết Thiết kế từng Màn chơi

3.3.1 Level 01: Nhập môn Hình học Không gian (Tutorial Basics)

- **Cấu trúc địa hình:** Kích thước nhỏ gọn 4×7 với 20 ô sàn đặc, không có lỗ hổng vực ở giữa sàn.
- **Mục tiêu thiết kế:** Màn khởi động giúp thuật toán làm quen với không gian trạng thái 3 chiều. Từ vị trí xuất phát $[1, 1]$ tới đích $[1, 6]$, khoảng cách Manhattan rất ngắn.
- **Thách thức thuật toán:** Tìm ra chuỗi 7 bước di chuyển cơ bản (`R-R-D-R-R-R-U`) sao cho khối gỗ chạm đích ở trạng thái đứng thẳng.

3.3.2 Level 02: Khúc eo Gấp khúc & Ngõ cụt (Narrow Passages)

- **Cấu trúc địa hình:** Dạng hành lang chữ S kéo dài 5×15, có những nút thắt chỉ rộng đúng 1 đến 2 ô vuông.
- **Mục tiêu thiết kế:** Kiểm tra khả năng tránh rơi vực của thuật toán khi di chuyển dọc theo bờ mép hẹp.
- **Thách thức thuật toán:** Ngăn chặn DFS đi lạc sâu vào các nhánh cụt phía trên bản đồ; nghiệm tối ưu đạt chính xác 12 bước.

3.3.3 Level 03: Vòng xoay Thay đổi Tư thế (Parity Inversion Island)

- **Cấu trúc địa hình:** Sàn hình chữ L ngược (7×13), ở giữa có một khoảng vực rộng ngăn cách trực tiếp giữa Start $[4, 1]$ và Goal $[0, 8]$.
- **Mục tiêu thiết kế:** Thử nghiệm tính toán chẵn-lẻ (Parity). Nếu đi thẳng đến trước cửa hố đích, khối gỗ sẽ bị lệch nhịp (đang ở thế nằm ngang, không thể lọt vào hố).
- **Thách thức thuật toán:** Buộc thuật toán phải chủ động đi vòng một chu trình (Loop) xung quanh mỏm sàn phía Nam để đảo ngược tư thế từ Nằm sang Đứng trước khi về đích (16 bước).

3.3.4 Level 04: Mê cung Diện tích Lớn (Open Field Labyrinth)

- **Cấu trúc địa hình:** Sàn rộng 7×16 với 64 ô đi được, nhiều mỏm nhô ra ngoài không gian.
- **Mục tiêu thiết kế:** Khảo sát hiện tượng bùng nổ tổ hợp (Combinatorial Explosion) của các thuật toán duyệt mù (BFS, IDS) trên không gian mở.
- **Thách thức thuật toán:** Các thuật toán có hàm Heuristic (A^*, Greedy) thể hiện ưu thế vượt trội khi cắt giảm số node duyệt từ hàng nghìn xuống còn vài chục node (17 bước tối ưu).

3.3.5 Level 05: Vùng cấm Ô cam (Fragile Introduction)

- **Cấu trúc địa hình:** Kích thước 4×11, chốt chặn giữa đường đi là 2 ô cam (Fragile tiles) nằm liền kề.

- **Mục tiêu thiết kế:** Kiểm tra tính toàn vẹn luật vật lý: khối gỗ đứng lên ô cam sẽ làm sập sàn.
- **Thách thức thuật toán:** Loại bỏ tất cả các nhánh tìm kiếm dẫn tới trạng thái đứng trên ô cam, chỉ chấp nhận nhánh lăn nằm trượt qua (11 bước).

3.3.6 Level 06: Bẫy Sập Sàn Đôi (Dual Fragile Trap)

- **Cấu trúc địa hình:** Bản đồ 5×12 có bố cục hình zíc-zắc, các ô cam được bố trí ngay tại góc cua bắt buộc.
- **Mục tiêu thiết kế:** Buộc thuật toán không chỉ lăn qua ô cam mà phải cua góc ngay trên đà di chuyển.
- **Thách thức thuật toán:** Khối gỗ phải chuyển đổi tư thế nằm dọc trước khi tiếp xúc với ô cam để không kích hoạt điều kiện sập sàn (15 bước).

3.3.7 Level 07: Thao trường Ô cam Dày đặc (Fragile Runway)

- **Cấu trúc địa hình:** Lưới 7×11 với 4 ô cam nằm liên tiếp tạo thành một dải băng mỏng.
- **Mục tiêu thiết kế:** Thử thách tính ổn định của hàm Heuristic khi đường chim bay ngắn nhất bị chặn hoàn toàn bởi sàn yếu.
- **Thách thức thuật toán:** Thuật toán Greedy dễ bị bẫy cục bộ; A^* tìm ra con đường vòng lượn khéo léo qua dải ô cam (17 bước).

3.3.8 Level 08: Vực thẳm Đa chướng ngại (Complex Fragile Obstacles)

- **Cấu trúc địa hình:** Bản đồ phức tạp 7×15 gồm 6 ô cam rải rác xen kẽ giữa các hành lang hẹp 1 ô.
- **Mục tiêu thiết kế:** Màn chơi tổng hợp độ khó của toàn bộ Nhóm 2.
- **Thách thức thuật toán:** Tìm kiếm không gian sâu; nghiệm tối ưu đòi hỏi 21 bước phối hợp chính xác giữa điều hướng và kiểm soát tư thế.

3.3.9 Level 09: Cầu nối Nhị phân và Công tắc Tròn (Soft Switch Mechanics)

- **Cấu trúc địa hình:** Hai hòn đảo độc lập ngăn cách bởi vực thẳm rộng 2 ô; ở giữa là cây cầu 101 đang đóng (trạng thái 0).
- **Mục tiêu thiết kế:** Giới thiệu cơ chế tương tác môi trường động (Dynamic Board State).
- **Thách thức thuật toán:** Thuật toán phải đi tới kích hoạt Công tắc Tròn (Soft Switch) tại đảo 1 để mở cầu 101, sau đó mới băng qua đảo 2 về đích (10 bước).

3.3.10 Level 10: Thử thách Áp lực Công tắc Chữ X (Heavy Switch Prerequisite)

- **Cấu trúc địa hình:** Kích thước 5×11, cầu 101 nối hai bờ tương tự Level 9 nhưng công tắc điều khiển là loại Chữ X (Heavy Switch).

- **Mục tiêu thiết kế:** Kiểm tra ràng buộc tư thế khắt khe: chỉ khi đứng thẳng toàn bộ trọng lượng lên nút chữ X thì cầu mới đóng/mở; lăn nằm đè lên nút sẽ hoàn toàn vô hiệu.
- **Thách thức thuật toán:** Cần thêm 4 bước di chuyển phụ để căn chỉnh nhịp bước chân sao cho khối gỗ tiếp cận nút chữ X ở đúng tư thế đứng thẳng (14 bước so với 10 bước ở Level 9).

3.3.11 Level 11: Kết hợp Cầu động và Ô cam Sập sàn (Bridges with Fragile Hazard)

- **Cấu trúc địa hình:** Đảo trung chuyển có ô cam nằm sát ngay chân cầu 101.
- **Mục tiêu thiết kế:** Tích hợp đồng thời hai cơ chế: vừa phải kích hoạt switch mở cầu, vừa phải né sập ô cam khi vừa đặt chân qua bờ bên kia.
- **Thách thức thuật toán:** Số bước tối ưu tăng vọt lên 24 bước do phải liên tục chuyển đổi tư thế để thích ứng với địa hình hỗn hợp.

3.3.12 Level 12: Bãi mìn Ô cam và Nút chữ X (Heavy Switch over Fragile Field)

- **Cấu trúc địa hình:** Bản đồ 5×12 chứa tới 8 ô cam bao quanh một nút chữ X nằm cô lập.
- **Mục tiêu thiết kế:** Cực hạn điều khiển: đứng thẳng trên nút chữ X nhưng xung quanh toàn là sàn dễ vỡ.
- **Thách thức thuật toán:** Lộ trình đòi hỏi thuật toán phải tìm ra điểm tựa ô cứng duy nhất để dựng đứng khối gỗ bấm nút rồi lập tức ngã nằm thoát hiểm (19 bước).

3.3.13 Level 13: Hệ thống Cầu kép Đối xứng (Dual Bridge Navigation)

- **Cấu trúc địa hình:** Ba hòn đảo tách rời, liên kết bởi 2 cây cầu riêng biệt (Cầu 101 và Cầu 102).
- **Mục tiêu thiết kế:** Đánh giá khả năng định tuyến chuỗi sự kiện tuần tự (Sequential Action Planning).
- **Thách thức thuật toán:** Bấm công tắc mở cầu thứ nhất $\rightarrow$ di chuyển sang đảo giữa $\rightarrow$ kích hoạt công tắc mở cầu thứ hai $\rightarrow$ tiến về đích (15 bước).

3.3.14 Level 14: Công tắc Phức hợp Đa chức năng (Multi-Switch Logic Puzzle)

- **Cấu trúc địa hình:** Bản đồ đa nhánh 8×15 gồm 2 switch (1 Soft Switch và 1 Heavy Switch) cùng 3 ô cam phân tán.
- **Mục tiêu thiết kế:** Bài toán tối ưu hóa có điều kiện: việc bấm sai thứ tự công tắc có thể dẫn đến việc đóng sập cây cầu duy nhất và gây kẹt cứng trạng thái (Deadlock).
- **Thách thức thuật toán:** Thuật toán phải nhận diện đúng thứ tự tác động công tắc để không rơi vào trạng thái bế tắc (19 bước).

3.3.15 Level 15: Phân thân Khối gỗ (Split Block Introduction)

- **Cấu trúc địa hình:** Địa hình gồm các đảo siêu nhỏ hẹp 1 ô, khối $1 \times 1 \times 2$ nguyên bản không thể nào xoay sở được nếu không tách đôi.
- **Mục tiêu thiết kế:** Khởi động cơ chế Phân thân: khối gỗ tách thành 2 khối con $1 \times 1 \times 1$, người chơi dùng phím chuyển đổi (`SPACE`) để điều khiển luân phiên từng khối.
- **Thách thức thuật toán:** Không gian trạng thái mở rộng thành tọa độ của cả 2 nửa khối gỗ; nghiệm tối ưu đạt 15 bước kết hợp hoàn hảo giữa tách, di chuyển và hợp thể.

3.3.16 Level 16: Mê cung Vực thẳm Cực hạn (The Marathon Chokepoint)

- **Cấu trúc địa hình:** Bố cục trải dài 5×14 nhưng đường đi uốn lượn liên tục qua 2 hệ thống cầu và bờ vực sâu không có rào chắn.
- **Mục tiêu thiết kế:** Màn chơi có độ dài đường đi lớn nhất trong toàn bộ hệ thống (46 bước tối ưu).
- **Thách thức thuật toán:** Thử thách độ bền của thuật toán tìm kiếm. Đây là màn chơi phân hóa rõ rệt nhất: trong khi A^* chỉ cần duyệt 283 node, thuật toán IDS phải mở rộng tới 9,933 node và DFS mất phương hướng duyệt gần 3,000 node.

3.3.17 Level 17: Phân đoạn Đảo Cô lập và Bẫy Ô cam (Island Hopping)

- **Cấu trúc địa hình:** Các hòn đảo nhỏ nằm so le được nối với nhau bằng cầu hẹp, bao bọc bởi 5 ô cam hiểm trở.
- **Mục tiêu thiết kế:** Khảo sát khả năng định hướng khi khoảng cách hình học ngắn nhưng khoảng cách trạng thái vật lý rất dài.
- **Thách thức thuật toán:** Thuật toán DFS bị bẫy lặp đi lặp lại tạo thành con đường dài tới 612 bước; trong khi BFS và A^* tìm ra con đường tối ưu chuẩn mực 21 bước.

3.3.18 Level 18: Chung kết Đỉnh cao – Tổng lực Cơ chế (Grand Finale Masterpiece)

- **Cấu trúc địa hình:** Đại mê cung kích thước 8×16 với 38 ô sàn, tích hợp đầy đủ mọi cơ chế: 2 ô cam, 2 công tắc tròn, 1 công tắc chữ X và 2 cây cầu đóng mở luân phiên.
- **Mục tiêu thiết kế:** Bài kiểm tra sát hạch cuối cùng (Capstone Challenge) cho toàn bộ 6 thuật toán AI.
- **Thách thức thuật toán:** Đòi hỏi thuật toán phải tính toán chính xác chuỗi kích hoạt mở cầu, vượt qua dải ô cam và căn chỉnh đúng tư thế đứng vào hố đen đích tại tọa độ $[6, 14]$ trong đúng 22 bước di chuyển tối ưu tuyệt đối.

3.4 Chỉ số đánh giá và cách đo

Các chỉ số thực nghiệm được đo lường trực tiếp thông qua hàm `solve()` của từng thuật toán để ghi nhận chính xác thời gian chạy (ms) và lượng RAM sử dụng (KiB). Mỗi thuật toán trên từng màn chơi được khởi tạo hoàn toàn độc lập để đảm bảo tính khách quan. Đồng thời, hệ thống thiết lập giới hạn thời gian chạy tối đa là 15.0 giây cho mỗi lượt thử; nếu vượt quá thời gian

này, thuật toán sẽ tự động dừng lại và ghi nhận kết quả là `TIMEOUT` nhằm tránh hiện tượng treo chương trình.

Bảng 3.2: Định nghĩa chỉ số benchmark và đánh giá chất lượng

Chỉ số	**Định nghĩa / Đơn vị**	**Quy ước diễn giải**
`Success`	Nghiệm hợp lệ được xác minh	Xác thực bằng cách replay chuỗi hành động trên bàn cờ vật lý đến trạng thái Goal. Thất bại nhận 0 điểm barem.
`Solution length` (S)	Số action trong đường đi	Tổng số bước di chuyển (bao gồm hành động `SPACE` khi phân tách khối *split*).
`Expanded nodes` (N)	Số trạng thái solver xử lý	Số trạng thái lấy ra khỏi Frontier để xét duyệt (loại bỏ các node lỗi thời trong Priority Queue; IDS cộng dồn qua các tầng lặp).
`Search time` (T)	Millisecond (ms)	Đo bằng `time.perf_counter()` trong `_search()`, loại trừ thời gian nạp JSON và khởi tạo ban đầu.
`Peak memory` (M)	KiB do `tracemalloc` ghi	Lượng bộ nhớ Python cấp phát đỉnh (*peak traced*) trong phiên tìm kiếm; không tính RSS hệ điều hành.
`Penalty Cost`	Điểm phạt rủi ro vật lý	Penalty $= S + 3.0B + 2.0V + 4.0F + 2.5K + 3.0R$ (S: bước, B: lùi, V: mép vực, F: ô vỡ, K: switch, R: cầu nguy hiểm).
`Trajectory Score`	Thang điểm barem $[0, 100]$	$\max\big(0.0,\ 100.0 - (\text{Penalty} - \text{Penalty}_{\min}) \times 2.5\big)$ với ngưỡng sai lệch tối đa $\Delta = 40.0$.
`Search Penetrance` (P)	$P = \frac{S}{N} \times 100\%$	Tỷ lệ thâm nhập tìm kiếm (S: số bước, N: số node mở rộng). Phản ánh độ sắc bén của Heuristic và mức độ bám sát đích.
`Suboptimality` (ρ)	$\rho = \frac{S}{S_{\min}}$	Hệ số lệch tối ưu; $\rho = 1.00$ là đường đi ngắn nhất tuyệt đối.
`Branching factor` (b^*)	Hệ số rẽ nhánh hiệu dụng	Giải phương trình cấp số nhân $N + 1 = \sum_{i=0}^{S} (b^*)^i$ bằng Binary Search trên đoạn $[1.0, 4.0]$ (chi tiết xem chú thích bên dưới).

Ghi chú về b^*: Chỉ số b^* được định nghĩa theo giáo trình của S. Russell và P. Norvig trong tài liệu [2] (Chương 3, Mục 3.6). Phương trình cấp số nhân $N + 1 = \sum_{i=0}^{S} (b^*)^i$ đo lường độ sắc bén của hàm Heuristic; b^* càng gần 1.0 thì thuật toán càng ít đi vào nhánh cụt. Giá trị được giải bằng thuật toán Binary Search trên đoạn $[1.0, 4.0]$.

3.5 Kết quả benchmark

Benchmark được thực thi trong môi trường thực tế mô tả ở Bảng 3.1 trên toàn bộ 18 màn chơi chính thức với 6 cấu hình thuật toán tìm kiếm (mỗi cặp được chạy lặp 5 lần để triệt tiêu sai số đo lường, tổng cộng 540 lượt chạy). Tất cả các lượt thử nghiệm đều tìm được nghiệm và nghiệm đều được phát lại hợp lệ tới trạng thái Goal. Bảng 3.3 trình bày giá trị trung bình của các chỉ số hiệu năng hệ thống và chất lượng lời giải sau khi lấy trung bình 5 lượt cho từng màn và tiếp tục lấy trung bình trên toàn bộ 18 màn chơi.

Bảng 3.3: Tổng hợp kết quả benchmark trung bình trên 18 màn chơi

Thuật toán	Thời gian (ms)	RAM (KiB)	Nodes	Số bước	Điểm (/100)	P (%)	ρ	b^*
BFS	139.74	116.95	348.8	17.8	100.0	14.87%	1.00	1.228
DFS	107.50	93.79	271.4	80.6	17.8	53.21%	4.43	1.032
IDS	1690.64	127.95	4278.7	17.8	100.0	1.61%	1.00	1.471
UCS (Rubric)	274.10	136.77	409.6	18.4	98.0	15.41%	1.03	1.217
A* (Rubric)	232.66	100.42	333.1	18.6	98.0	17.70%	1.04	1.199
GBFS (Greedy)	29.09	16.06	49.6	20.8	85.3	45.21%	1.17	1.089

Ghi chú: UCS và A dùng chi phí Rubric* $(w_{step}, w_{back}, w_{void}, w_{frag}, w_{switch}, w_{bridge}) = (1.0, 3.0, 2.0, 4.0, 2.5, 3.0)$ *với* $\alpha = 1.0$*; GBFS dùng heuristic* $(w_{ori}, w_{split}, w_{edge}, w_{frag}) = (1.0, 1.0, 0.0, 0.0)$*. Số liệu được tính bằng trung bình 5 lượt chạy trên 18 màn chơi.*

Nhận xét và phân tích kết quả:

1. **Tính tối ưu về số bước đi:** BFS và IDS luôn bảo đảm tìm được lời giải ngắn nhất tuyệt đối trên mọi màn chơi (trung bình 17.8 bước, tỉ số dưới tối ưu $\rho = 1.00$ và đạt điểm chất lượng tuyệt đối 100.0/100).
2. **Đánh đổi giữa chi phí an toàn và số bước đi:** Do UCS và A* tối thiểu hóa hàm chi phí có phạt các trạng thái nguy hiểm (đi lùi, đứng gần mép vực, đè lên ô mỏng), quỹ đạo của hai thuật toán này chấp nhận đi đường vòng an toàn ở một số màn chơi phức tạp, dẫn đến số bước trung bình là 18.4 và 18.6 bước ($\rho \leq 1.04$, điểm an toàn đạt 98.0/100). Đổi lại, A* với heuristic định hướng đã cắt giảm được 18.7% số node mở rộng so với UCS (333.1 so với 409.6 nodes) và giảm 15.1% thời gian thực thi.
3. **Tốc độ và hiệu quả bộ nhớ:** GBFS (Greedy) thể hiện ưu thế vượt trội về tốc độ (chỉ mất 29.09 ms) và dung lượng RAM cực thấp (16.06 KiB), chỉ cần mở rộng 49.6 nodes nhờ độ thâm nhập trạng thái rất cao ($P = 45.21\%$). Tuy nhiên, tính tham lam khiến nghiệm của GBFS dài hơn (20.8 bước, $\rho = 1.17$).
4. **Hạn chế của DFS và IDS:** DFS đi rất sâu vào các chu trình không gian trạng thái trước khi tìm thấy đích, dẫn đến số bước trung bình bùng nổ lên tới 80.6 bước ($\rho = 4.43$) và điểm quỹ đạo rơi xuống mức thấp kỷ lục (17.8/100). Trong khi đó, IDS bảo toàn tính tối ưu bước đi nhưng phải trả giá bằng việc liên tục duyệt lặp lại cây tìm kiếm, đẩy số node mở rộng lên 4278.7 nodes và thời gian chạy trung bình lên tới 1690.64 ms.

3.5.1 Độ dài lời giải theo từng level

Bảng 3.4 trình bày số lượng hành động (*solution length / actions*) tìm được trên từng màn chơi đối với các thuật toán đã triển khai. Kết quả thực nghiệm cho thấy DFS có xu hướng đi vòng và

trả về đường đi dài bất thường ở các màn mê cung phức tạp (đặc biệt Level 17 lên tới 612 bước), trong khi BFS và IDS luôn đảm bảo tìm ra đường đi ngắn nhất tối ưu trên mọi màn chơi.

Bảng 3.4: Số action giải được theo từng level và solver (18 màn chơi)

Level	BFS	DFS	IDS	UCS	A*	GBFS
01 – Basic	7	21	7	7	7	7
02 – Fragile	12	55	12	13	13	13
03 – Soft bridge	16	35	16	16	16	16
04 – Heavy bridge	17	77	17	17	17	18
05 – Fragile Path	11	53	11	11	11	11
06 – Combined	15	35	15	15	15	22
07 – Fragile Detour	17	29	17	17	17	18
08 – Toggle Bridge	21	53	21	22	24	25
09 – Binary Bridges	10	42	10	10	10	10
10 – Precision Bridges	14	64	14	14	14	26
11 – Dynamic Path	24	43	24	24	24	24
12 – Fragile & Heavy	19	33	19	21	21	33
13 – Island Hopping	15	38	15	15	15	15
14 – Precision Gauntlet	19	53	19	19	19	20
15 – Split Intro	15	15	15	15	15	17
16 – Split Switches	46	104	46	46	46	48
17 – Complex Split	21	612	21	24	24	29
18 – Grand Finale	22	89	22	26	26	23
Trung bình (Mean)	**17.8**	**80.6**	**17.8**	**18.4**	**18.6**	**20.8**

Ghi chú: UCS và A* sử dụng hàm chi phí chuẩn hóa Rubric với bộ trọng số ($w_{\text{step}} = 1.0$, $w_{\text{back}} = 3.0$, $w_{\text{void}} = 2.0$, $w_{\text{frag}} = 4.0$, $w_{\text{switch}} = 2.5$, $w_{\text{bridge}} = 3.0$) và $\alpha = 1.0$ cho A*.

Phân tích độ dài đường đi giữa các thuật toán:

- **Độ tối ưu của BFS và IDS:** Luôn đạt số bước tối thiểu trên cả 18 level (trung bình 17.8 bước).
- **Hành vi an toàn của UCS và A*:** Tại 13/18 màn chơi, UCS và A* tìm ra đường đi ngắn nhất trùng với BFS. Tuy nhiên, tại các màn có địa hình hiểm trở và nhiều ô vỡ (Level 02, 08, 12, 17, 18), hai thuật toán này chủ động chọn đường đi vòng an toàn thêm từ 1 đến 4 bước (ví dụ: Level 02 tốn 13 bước so với 12 của BFS; Level 12 tốn 21 bước so với 19 bước; Level 17 tốn 24 bước so với 21 bước; Level 18 tốn 26 bước so với 22 bước) để tránh mức phạt cao khi đứng trên ô mỏng (*fragile penalty*) và đi sát mép vực (*void penalty*). Riêng ở Level 08, A* chọn một nhánh an toàn khác có độ dài 24 bước so với 22 bước của UCS.
- **Sự phân hóa của GBFS:** Do chỉ tập trung vào heuristic khoảng cách mà không xét

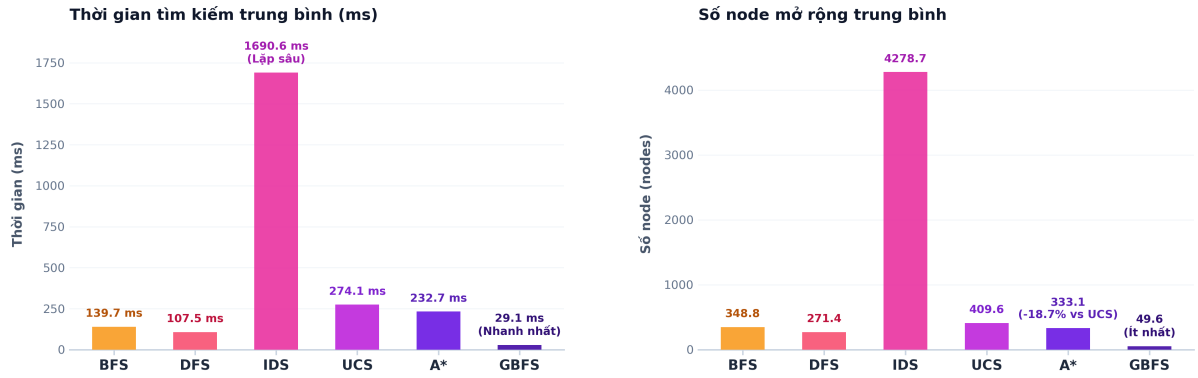
chi phí thực tế, GBFS thường bị vướng vào các đường vòng cục bộ ở các màn có cơ chế chuyển đổi cầu phức tạp (như Level 06 tốn 22 bước so với 15 bước của BFS; Level 10 tốn 26 bước so với 14 bước; Level 12 tốn 33 bước so với 19 bước).

- **Sự bùng nổ của DFS:** DFS hoàn toàn không đảm bảo tính tối ưu bước đi. Đặc biệt tại Level 17 (màn chia tách khối phức tạp), DFS duyệt sâu vào các nhánh chu trình dẫn tới số bước lên tới 612 bước (gấp 29.1 lần so với nghiệm tối ưu 21 bước của BFS).

3.5.2 So sánh tài nguyên tính toán

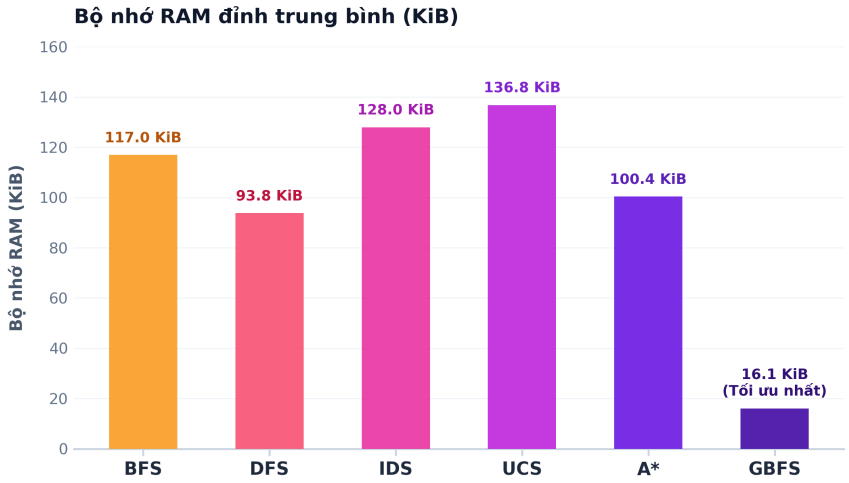
Hình 3.1 và Hình 3.2 trực quan hóa các chỉ số hiệu năng trung bình của 6 thuật toán trên toàn bộ 18 màn chơi chính thức:

- **Thời gian và số node mở rộng (Hình 3.1a, 3.1b):** GBFS là thuật toán chạy nhanh nhất (29.1 ms) và mở ít trạng thái nhất (49.6 nodes) do chiến lược tham lam bám sát đích; tuy nhiên tốc độ này đánh đổi lại độ dài nghiệm dài hơn 17% so với tối ưu. Ngược lại, A* thể hiện sự cân bằng tốt nhất khi giảm được 18.7% số node mở rộng so với UCS (333.1 so với 409.6 nodes) nhờ hàm Heuristic dẫn đường. Thuật toán IDS do phải liên tục duyệt lặp lại cây tìm kiếm ở mỗi ngưỡng độ sâu nên số node (4278.7 nodes) và thời gian chạy (1690.6 ms) bùng nổ cao nhất.
- **Mức độ tiêu thụ bộ nhớ RAM (Hình 3.2):** GBFS tiêu tốn ít RAM nhất (16.1 KiB), trong khi UCS và IDS chiếm dụng bộ nhớ cao nhất (136.8 KiB và 128.0 KiB) do chi phí duy trì hàng đợi ưu tiên và ngăn xếp duyệt sâu. Các chỉ số được đo đạc chính xác bằng module `tracemalloc` trên heap của Python.



(a) Thời gian tìm kiếm trung bình (ms). (b) Số node mở rộng trung bình.

Hình 3.1: So sánh tài nguyên tìm kiếm. Mỗi cột là giá trị trung bình trên 18 màn chơi sau khi lấy trung bình của 5 lượt lặp độc lập cho mỗi màn.



Hình 3.2: Bộ nhớ RAM đỉnh trung bình (KiB) theo thuật toán. Chỉ số là đỉnh cấp phát Python được truy vết thông qua thư viện `tracemalloc`, không phải RSS hệ điều hành.

3.6 Giới hạn thực nghiệm và tính tái lập

- **Phạm vi đo lường bộ nhớ:** Thư viện `tracemalloc` chỉ ghi nhận lượng bộ nhớ đỉnh cấp phát trực tiếp cho việc lưu trữ các trạng thái và nút tìm kiếm của thuật toán trên Python heap. Chỉ số này không phản ánh toàn bộ dung lượng RAM của tiến trình hệ điều hành (RSS) hay tài nguyên chiếm dụng của hệ thống giao diện đồ họa 3D.
- **Kiểm thử giao diện đồ họa (GUI):** Quy trình benchmark tự động chỉ tập trung vào hiệu năng của các thuật toán tìm kiếm trên môi trường dòng lệnh (headless). Chức năng giao diện người dùng và tương tác đồ họa 3D được xác minh độc lập thông qua quá trình trải nghiệm thực tế và video demo đính kèm.

Toàn bộ dữ liệu chi tiết của 540 lượt chạy được lưu trữ tại tập tin `benchmark_results.csv` và tóm tắt tại `benchmark_summary.csv`. Kết quả thực nghiệm có thể được kiểm chứng và tái lập độc lập thông qua tập lệnh `benchmark.py` trong thư mục gốc của dự án.

3.7 Quy trình lựa chọn tham số cho UCS, A* và GBFS

Thí nghiệm sử dụng 18 level Bloxorz dạng JSON và gọi mô hình chuyển trạng thái của dự án để sinh các chuyển tiếp hợp lệ. Nhóm dùng một hàm đánh giá đa tiêu chí được định nghĩa trước và kiểm tra khả năng khái quát bằng grouped cross-validation, trong đó đơn vị chia fold là level chứ không phải từng trạng thái hoặc từng trajectory. Phần tuning được chạy bằng implementation thực nghiệm riêng nhưng dùng chung mô hình trạng thái và hàm chuyển.

Hàm đánh giá kết hợp sáu tiêu chí: số lần lăn U, D, L, R; số lần lăn đảo chiều ngay lập tức; mức phơi nhiễm mép vực; số ô fragile mới đi vào; số cờ cầu thay đổi; và số ô cầu mới đi vào. Các giá trị được chuyển thành thứ hạng phân vị trong từng level để hạn chế việc các tiêu chí có thang đo khác nhau chi phối kết quả. Trọng số ưu tiên được cố định theo tỷ lệ $(1, 3, 2, 4, 2.5, 3)$ tương ứng với sáu tiêu chí nêu trên. Lời giải thất bại hoặc không hợp lệ nhận mức phạt lớn hơn mọi lời giải hợp lệ. Vì vậy, đây là quy trình tinh chỉnh kỹ thuật theo rubric của nhóm, không phải học có giám sát từ sở thích người dùng độc lập.

UCS: grid search cho hàm chi phí C. Trọng số bước w_{step} được cố định ở 1.0 để tránh sự không xác định về thang đo khi đồng thời nhân tất cả hệ số chi phí với một hằng số. Năm trọng số phạt được khảo sát trên lưới $3^5 = 243$ tổ hợp; các mức khảo sát lần lượt là $w_{\text{back}} \in \{0, 3, 5\}$, $w_{\text{void}} \in \{0, 2, 4\}$, $w_{\text{frag}} \in \{0, 4, 8\}$, $w_{\text{switch}} \in \{0, 2.5, 5\}$ và $w_{\text{bridge}} \in \{0, 3, 5\}$. Nhóm so sánh các cấu hình với ba baseline: rubric khởi đầu $(3, 2, 4, 2.5, 3)$; baseline không có khoản phạt; và baseline đồng trọng số $(1, 1, 1, 1, 1)$. Tổng cộng có 244 cấu hình được xem xét nếu tính cả baseline đồng trọng số nằm ngoài lưới chính.

Sử dụng sáu fold, mỗi fold giữ lại ba level để đánh giá và dùng 15 level còn lại để lựa chọn cấu hình. Cấu hình tốt nhất trên toàn bộ 18 level có mean rank loss 0.066618, nhỉnh hơn một chút so với baseline rubric 0.067813 theo tiêu chí trên toàn tập. Tuy nhiên, cải thiện trung bình trên các fold giữ lại so với rubric là -0.002743; dấu âm thể hiện cấu hình thay thế không cải thiện trung bình trên dữ liệu giữ lại. Do quy tắc chọn yêu cầu cải thiện ngoài mẫu, nhóm giữ lại bộ trọng số rubric ban đầu $(3, 2, 4, 2.5, 3)$ thay vì tuyên bố cấu hình grid-search toàn dữ liệu là tốt hơn.

A*: kiểm tra hệ số heuristic α. A* sử dụng chính hàm chi phí C được lựa chọn cho UCS, nhưng ưu tiên nút theo $f(N) = g(N) + \alpha h_0(S)$. Nhóm khảo sát $\alpha \in \{0, 0.25, 0.5, 0.75, 1.0\}$. Trong mỗi fold, các ứng viên trước hết phải tạo lời giải hợp lệ và có tổng chi phí trùng với UCS trên các level được kiểm tra; sau đó chọn hệ số làm giảm số trạng thái mở rộng trung bình nhiều nhất. Cả sáu fold đều chọn $\alpha = 1.0$, cũng là giá trị được chọn khi đánh giá trên toàn bộ 18 level. Trong phép dò hệ số, số trạng thái mở rộng trung bình giảm từ 1285.8 tại $\alpha = 0$ xuống 989.7 tại $\alpha = 1.0$, tương đương khoảng 23.0%. Do giá trị cuối cùng là 1.0, công thức heuristic nền không bị thay thế; thí nghiệm chỉ xác nhận rằng không nên giảm hệ số heuristic trong tập benchmark này theo tiêu chí đã chọn.

Về giới hạn lý thuyết: kiểm tra heuristic nền bằng chi phí chuyển động cơ sở (lăn có chi phí 1, đổi quyền điều khiển có chi phí 0) không phát hiện vi phạm admissibility/consistency ở level 1–14, nhưng ghi nhận vi phạm trên các level Split 15–18. Mặc dù A* với các hệ số đã thử đều cho lời giải hợp lệ có tổng chi phí khớp UCS trên 18 level, quan sát này chỉ là bằng chứng thực nghiệm trên bộ test hiện tại, không thay thế chứng minh tối ưu tổng quát cho mọi trạng thái Split.

Bảng 3.5 trình bày kết quả của toàn bộ lưới hệ số. Thời gian ở bảng là thời gian tìm kiếm trên đồ thị đã được lưu đệm (cached graph search), nên được dùng như chỉ số so sánh nội bộ trong thí nghiệm này.

Bảng 3.5: Kết quả khảo sát hệ số heuristic α của A* trên 18 level

α	Mean expanded nodes	Median expanded nodes	Mean time (ms)	Kiểm tra lời giải
0.00	1285.8	216.0	10.80	Hợp lệ; tổng chi phí bằng UCS trên 18 level
0.25	1211.4	211.0	7.90	Hợp lệ; tổng chi phí bằng UCS trên 18 level
0.50	1136.3	209.5	7.57	Hợp lệ; tổng chi phí bằng UCS trên 18 level
0.75	1053.6	199.5	6.82	Hợp lệ; tổng chi phí bằng UCS trên 18 level
1.00	989.7	187.0	6.56	Hợp lệ; tổng chi phí bằng UCS trên 18 level

Ghi chú: Sáu fold grouped cross-validation đều chọn $\alpha = 1.0$. Số node được làm tròn đến một chữ số thập phân; thời gian là trung bình cached graph-search time. Việc khớp chi phí UCS là kết quả quan sát trên bộ benchmark này, không thay thế chứng minh tối ưu tổng quát.

GBFS: grid search cho heuristic riêng. GBFS không sử dụng hàm chi phí đường đi C trong khóa ưu tiên; do đó, việc khảo sát của thuật toán này là tinh chỉnh heuristic chứ không phải tinh chỉnh chi phí UCS. Nhóm khảo sát 256 cấu hình từ các tổ hợp $w_{\text{orientation}} \in \{0, 0.5, 1, 1.5\}$, $w_{\text{split}} \in \{0, 1, 2, 3\}$, $w_{\text{edge}} \in \{0, 0.5, 1, 2\}$ và $w_{\text{fragile}} \in \{0, 1, 2, 4\}$. Cách đánh giá và sáu fold theo nhóm level giống quy trình rubric nêu trên.

Cấu hình tốt nhất khi tính trên toàn bộ dữ liệu có mean rank loss 0.104238, thấp hơn mức 0.122398 của heuristic ban đầu $(1, 1, 0, 0)$. Tuy nhiên, các cấu hình được chọn riêng từ tập huấn luyện có mean loss trên các level giữ lại là 0.122960, cao hơn mức 0.122398 của baseline. Vì vậy, nhóm giữ heuristic GBFS ban đầu với $(w_{\text{orientation}}, w_{\text{split}}, w_{\text{edge}}, w_{\text{fragile}}) = (1, 1, 0, 0)$. Kết quả này không có nghĩa là heuristic chưa từng được thử tinh chỉnh; nó cho thấy grid search không đạt cải thiện ổn định theo grouped cross-validation.

Bảng 3.6: Tổng hợp kết quả lựa chọn tham số

Thuật toán	**Không gian khảo sát**	**Kết quả định lượng**	**Quyết định cuối cùng**
UCS	243 cấu hình trên lưới, cộng baseline đồng trọng số	Loss tốt nhất trên toàn bộ dữ liệu là 0.066618, so với 0.067813 của baseline rubric. Tuy nhiên, cải thiện held-out trung bình là -0.002743.	Giữ baseline rubric $(3, 2, 4, 2.5, 3)$ với $w_{\text{step}} = 1$, vì ứng viên không cải thiện trên các fold giữ lại.
A*	5 giá trị của hệ số α	Mean expanded nodes giảm từ 1285.8 ($\alpha = 0$) xuống 989.7 ($\alpha = 1$), giảm khoảng 23.0%. Sáu fold đều chọn $\alpha = 1.0$.	Chọn $\alpha = 1.0$. Đây là lựa chọn hệ số cho heuristic nền, không phải đề xuất một heuristic mới.
GBFS	256 cấu hình heuristic	Loss tốt nhất trên toàn bộ dữ liệu là 0.104238, so với 0.122398 của baseline. Tuy nhiên, loss held-out trung bình của các cấu hình được chọn theo fold là 0.122960.	Giữ heuristic baseline $(1, 1, 0, 0)$ vì cấu hình thay thế không cải thiện ổn định trên các level giữ lại.

Ghi chú: Với UCS và GBFS, loss càng thấp càng tốt. Giá trị held-out improvement âm biểu thị kết quả không tốt hơn baseline. Chỉ số A* được đánh giá bằng số trạng thái mở rộng, không cùng thang đo với loss. Các kết luận giới hạn trong 18 level, lưới tham số và rubric đã khảo sát.

Phạm vi. Các trọng số UCS và GBFS được lựa chọn theo tiêu chí kỹ thuật có quy định trước, nhưng tiêu chí này vẫn là phiếu tự đánh giá (rubric) do nhóm thiết kế, không phải nhãn chất lượng độc lập. Sáu fold cung cấp một kiểm tra độ ổn định theo level, không phải một tập kiểm thử ngoài hoàn toàn độc lập, vì cả 18 level đều tham gia giai đoạn phát triển và lựa chọn cuối cùng. Do đó, báo cáo diễn giải các hệ số là cấu hình phù hợp nhất trong phạm vi lưới, rubric và benchmark đã thử, không khẳng định là tối ưu phổ quát.

CHƯƠNG 4

Kế hoạch và phân công công việc

4.1 Phân công nhiệm vụ

Các thành viên được phân công theo phần việc chính để bảo đảm tiến độ xây dựng trò chơi, cài đặt thuật toán tìm kiếm và đánh giá thực nghiệm. Bảng 4.1 tóm tắt trách nhiệm của từng thành viên; các phần việc có liên quan được kiểm thử và hỗ trợ chéo trước khi tích hợp.

MSSV	Thành viên	Nhiệm vụ chính	Mức đóng góp
1	Mai Hoàng Hiệp Hiệp	Cài đặt UCS, thiết kế hàm chi phí và tham gia thực nghiệm tinh chỉnh trọng số, phân tích kết quả.	100%
2	Nguyễn Hoài Phước	Xây dựng mô hình trạng thái và hành động, cài đặt BFS; phụ trách phần giao diện/hiển thị và hỗ trợ tích hợp trò chơi.	100%
3	Nguyễn Đức Tài	Cài đặt DFS và IDS; phát triển bộ điều khiển, thao tác chơi và chức năng phát lại lời giải.	100%
4	Phan Nguyễn Gia Huy	Cài đặt GBFS; xây dựng bộ kiểm thử, tự động chạy benchmark và tổng hợp số liệu so sánh.	100%
5	Phan Thế Phong	Cài đặt A*; phát triển các cơ chế nâng cao của Bloxorz như ô fragile, cầu, công tắc và trạng thái split/merge; hỗ trợ kiểm thử tích hợp.	100%

Bảng 4.1: Phân công nhiệm vụ chính của các thành viên

CHƯƠNG 5

Kết luận

Đồ án Bloxorz Solver triển khai và đối chiếu nhiều thuật toán tìm kiếm trên cùng mô hình trạng thái và hàm chuyển, bao gồm các cơ chế địa hình, công tắc, cầu và tách khối. Việc chuẩn hóa trạng thái cùng quy tắc chuyển tiếp giúp các thuật toán được kiểm thử trên cùng một không gian bài toán; đồng thời, các thử nghiệm cho thấy kết luận về hiệu năng cần gắn với hàm chi phí, heuristic, cách biểu diễn trạng thái và phạm vi benchmark cụ thể.

5.1 Kết quả và đóng góp trong phạm vi đồ án

Có thể tóm tắt kết quả thành ba nhóm. Thứ nhất, benchmark chức năng của dự án trên 10 level, 9 cấu hình solver và 5 lần lặp mỗi cặp level–solver ghi nhận 450/450 lượt chạy trả về lời giải được phát lại hợp lệ đến đích. Đây là bằng chứng cho các trường hợp đã kiểm tra, không phải chứng minh rằng mọi level Bloxorz đều được giải thành công. Bộ test này vẫn nhỏ; các kết quả thời gian ở mức rất thấp nên chủ yếu mang tính mô tả.

Thứ hai, trong đợt thực nghiệm bổ sung về lựa chọn tham số, nhóm khảo sát 18 level với grouped cross-validation theo level. Với UCS, 243 cấu hình trọng số trong lưới chính và các baseline được so sánh theo một rubric đa tiêu chí. Cấu hình tốt nhất khi chấm trên toàn bộ 18 level có mean rank loss thấp hơn nhẹ so với baseline rubric, nhưng mức cải thiện không giữ được trên các level giữ lại; vì vậy, nhóm giữ bộ trọng số ban đầu $(3, 2, 4, 2.5, 3)$. Đây là kết quả có giá trị thực nghiệm: không chọn cấu hình chỉ vì nó cho điểm tốt hơn trên toàn bộ dữ liệu đã xem.

Với A*, nhóm giữ nguyên hàm chi phí đã chọn cho UCS và khảo sát hệ số α trong $h_\alpha(S) = \alpha h_0(S)$ trên lưới $\{0, 0.25, 0.5, 0.75, 1.0\}$. Cả sáu fold đều chọn $\alpha = 1.0$. Trên bộ 18 level, số trạng thái mở rộng trung bình giảm từ khoảng 1285.8 ở $\alpha = 0$ xuống khoảng 989.7 ở $\alpha = 1.0$, tương đương khoảng 23% theo lần chạy thực nghiệm này. Tất cả giá trị α được thử đều trả về đường đi hợp lệ có tổng chi phí khớp với UCS trên 18 level đã kiểm tra. Tuy nhiên, kết quả so khớp thực nghiệm không thay thế chứng minh tính tối ưu cho mọi trạng thái.

Với GBFS, nhóm khảo sát $4^4 = 256$ tổ hợp trọng số heuristic. Cấu hình được giữ là $(w_{\text{orientation}}, w_{\text{split}}, w_{\text{edge}}, w_{\text{fragile}}) = (1, 1, 0, 0)$, trùng với heuristic ban đầu. Một cấu hình đạt rank loss tốt hơn khi chấm trên toàn bộ dữ liệu, nhưng hiệu quả đó không được duy trì trên các level giữ lại. Vì vậy, kết quả hiện có chưa ủng hộ việc bổ sung các khoản phạt mép vực hoặc ô fragile vào heuristic GBFS.

Tài liệu tham khảo

[1] Faculty of Information Technology, University of Science, VNU-HCM. *CSC14003 – Introduction to Artificial Intelligence: Project 1, Bloxorz Solver*.

[2] S. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. 4th ed., Pearson, 2020.

[3] Coolmath Games. *Bloxorz*. https://www.coolmathgames.com/0-bloxorz.

[4] Python Software Foundation. *heapq – Heap queue algorithm*. https://docs.python.org/3/library/heapq.html.

[5] Python Software Foundation. *tracemalloc – Trace memory allocations*. https://docs.python.org/3/library/tracemalloc.html.

[6] Python Software Foundation. *time – Time access and conversions*. https://docs.python.org/3/library/time.html.

[7] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.